Chapter 16

Experimentation and Bandits

After reading this chapter, you will be able to:

- Design a trustworthy A/B test for a recommendation surface: choose the randomization and analysis units, an overall evaluation criterion with guardrails, the minimum detectable effect, the sample size and duration, and a decision rule written down before launch, then validate the setup with A/A tests and sample ratio mismatch checks.
- Increase experiment sensitivity with variance reduction (CUPED, CUPAC, post-stratification, and triggering), analyze ratio metrics correctly with the delta method, explain why repeatedly checking results inflates false positives, and choose between fixed-horizon, group sequential, and always-valid tests.
- Recognize when a recommender experiment violates the stable unit treatment value assumption through shared training data, shared inventory or budgets, social connections, or supply-side response, and select a cluster, two-sided, budget-split, switchback, data-diverted, or user-corpus co-diverted design whose bias, variance, and engineering cost fit the situation.
- Identify treatment effects that change over time (novelty, primacy, and user learning) and plan long-term measurement with holdbacks, post-period analysis, and surrogate or learned proxy metrics.
- Use team-draft interleaving to compare rankers with far less traffic than an A/B test, place it in a funnel with A/B testing, and state what interleaving cannot measure.
- Formulate an exploration problem as a multi-armed or contextual bandit, implement epsilon-greedy, UCB1, Thompson sampling, and LinUCB, explain regret and the logging requirements of exploration, and decide when a bandit is preferable to an A/B test.
- Estimate a new policy's value from logged exploration data with replay, inverse propensity scoring, self-normalized inverse propensity scoring, the direct method, and doubly robust estimators, diagnose support and variance failures, and decide how much weight an offline estimate deserves before an online test.

16.1 The Winner That Lost

The following account is a composite, constructed for illustration from failure patterns that the experimentation literature documents repeatedly. It describes no particular company, and it is built on the music-streaming pipeline that this book has assembled: the rankers of Chapters 6 and 7, the value function of Chapter 8, and the services deployed in Chapter 13.

A team trains a new multi-task ranker for the personalized home shelf. On the temporal evaluation split it beats the production ranker, the champion, on NDCG@10, and the margin holds across user activity slices and for cold-start items. The candidate then passes every release gate from Chapter 15. Shadow evaluation finds no missing features and no encoder mismatch, the canary shows no rise in errors or

latency, and the score distributions look healthy. The team launches an A/B test that sends half of all users to the new ranker and keeps half on the champion, with click-through rate on the shelf as its headline metric.

Everyone wants the result, so the team opens the experiment dashboard every morning. On the fifth day the difference in click-through rate crosses the conventional significance threshold, the treatment arm is ahead, and the team declares a winner and ramps the new ranker to all traffic. The launch review is short, because offline metrics, release gates, and an online experiment all agree.

Several months later, the evidence turns. The company keeps a small group of users on the previous experience for months after every major launch, a long-term holdback that Section 16.6.2 defines properly. When an analyst compares that group with everyone else, thirty-day retention, the company's North Star metric, shows no difference at all. Users served by the new ranker also listen to a visibly narrower slice of the catalog. The winner did not win.

The story contains three common errors of inference, each the subject of a later section. The first is optional stopping: checking a test daily and stopping at the first significant reading inflates the false positive rate far above its nominal level, so the day-five result may have been noise (Section 16.4.5). The second is novelty: a changed mix of tracks draws curiosity clicks that fade as users grow used to it, so an early lift can overstate, or even reverse, the lasting effect (Section 16.6.1). The third is a proxy metric that diverged from the North Star. Click-through rate rewarded familiar, easy-to-click tracks, while retention depends on users continuing to find music worth returning for (Section 16.3.4).

The offline gain misled the team for a deeper reason. NDCG on held-out logs answers a predictive question: does the new model rank the items users interacted with above the items they did not, given the exposure the old system chose? A launch decision needs the answer to a causal question: what will happen to users and to the business if this system replaces the current one? The two can diverge for all the reasons Chapter 3 described, including position bias, exposure bias, and feedback loops. Booking.com reported this divergence directly. Across its customer-facing machine learning models, each validated by a randomized controlled trial, gains in offline model performance were not a reliable guide to gains in business value (Bernardi et al., 2019). This chapter supplies the online side of the offline-online comparison that Section 3.4.1 recommended tracking.

The release gates were not wrong either: a canary asks whether a release is safe to keep rolling out (Section 15.9.5), and the new ranker was safe. Whether it was better was a separate question, answered badly.

The rest of the chapter covers trustworthy test design, detecting small effects in heavy-tailed data, interference between users, effects that change over months, cheaper ranker comparisons, adaptive allocation, and evaluating undeployed policies from existing logs.

16.2 From Offline Evidence to Causal Evidence

A small amount of causal vocabulary states what an experiment estimates, why randomization makes the estimate valid, and which assumptions the rest of the chapter keeps returning to.

16.2.1 Potential Outcomes in One Page

An experiment applies a treatment to units. A unit is whatever receives the treatment and generates outcomes: in most recommendation experiments it is a user or account, although it can be a device, a session, a single request, an item, or a geographic market. The treatment is the change being evaluated,

such as a new ranker, a different value-function weighting, or a redesigned shelf, and the control is the current experience.

For each unit i, imagine two outcomes. $Y_i(1)$ is the outcome the unit would produce if treated, perhaps the number of tracks a user completes over two weeks, and $Y_i(0)$ is the outcome the same unit would produce under control. These are called potential outcomes, and the effect of the treatment on unit i is the difference $Y_i(1) - Y_i(0)$. Only one of the two can ever be observed, because a user cannot both receive the new ranker and not receive it over the same two weeks, so experiments target an average instead. The average treatment effect is

$$\tau = \mathbb{E}[Y_i(1) - Y_i(0)],$$

the mean difference across the population of units.

Random assignment is what makes τ estimable. If a fair coin decides which units are treated, the treated and control groups differ only by chance in everything that affects outcomes, including taste, activity level, device, and country, whether those characteristics are measured or not. The control group's average outcome then estimates what the treated group would have produced without treatment, and the simple difference in group means, $\bar{Y}_T - \bar{Y}_C$, is an unbiased estimate of τ. No observational comparison offers that guarantee, because users who happen to receive a change usually differ systematically from those who do not.

The guarantee rests on an assumption called the stable unit treatment value assumption, or SUTVA, which has two parts. The first is no interference: a unit's outcome depends only on its own assignment, never on whether other units were treated. The second is no hidden versions of treatment: "treated" means one well-defined thing for every unit. Recommenders strain both parts. Treated and control users may share a model that trains on everyone's interactions, compete for the same inventory, or send each other content, which violates the first part. Brennan et al. (2025) show that shared training data also violates the second part, because the "control" model is no longer the model that would exist if the treatment had never been launched. Section 16.5 takes up these failures and the designs that repair them. Kohavi et al. (2020) remain the standard practitioner reference for the experimental framing used throughout this chapter.

16.2.2 The Evaluation Ladder

A randomized experiment on live users is the most direct evidence about a causal question, and also the most expensive. It consumes traffic that other experiments could use, takes weeks, and exposes real users to a variant that may be worse. Teams therefore gather cheaper kinds of evidence first and filter candidates as they go.

Figure 16.1 arranges five kinds of evidence as rungs. Offline metrics computed on held-out logs, the subject of Section 3.6, ask whether a candidate ranks logged interactions better. They cost almost nothing and say little about causal impact. Off-policy evaluation, covered in Section 16.9, uses logs that were collected with deliberate randomization to estimate what a new policy would have earned had it been serving, with no new user exposure. Interleaving, covered in Section 16.7, merges the output of two rankers into a single list for each user and infers which ranker users prefer from where they click; this is an evaluation method and differs from the blending of sponsored and organic results that Section 9.3.5 also called interleaving. An A/B test measures how user and business metrics change, and by how much, over a period of weeks. A long-term holdback keeps a small group on the old experience long after launch to ask whether the effect lasts.

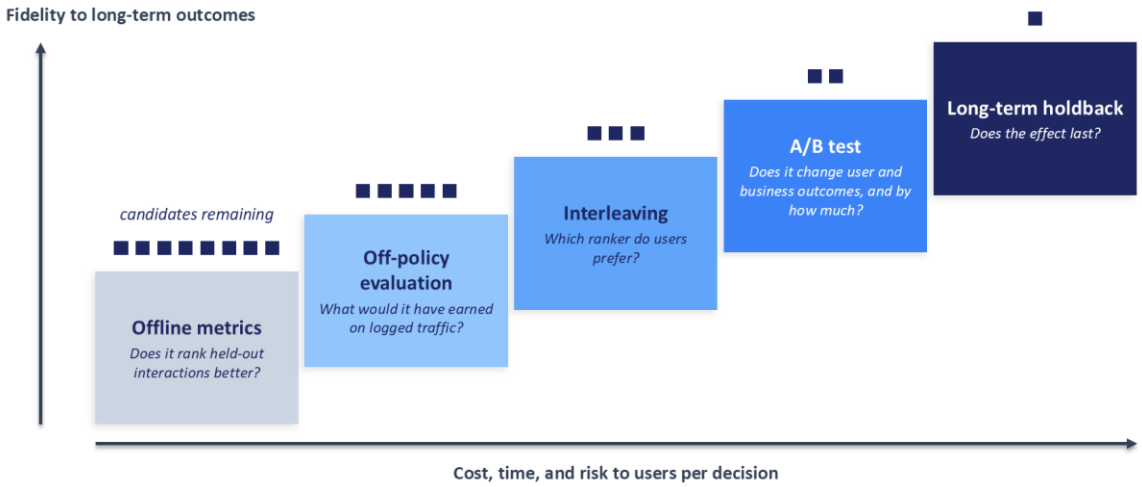


Figure 16.1: The evaluation ladder. Five kinds of evidence answer different questions at increasing cost, time, and risk to users, and with increasing fidelity to long-term outcomes. Candidates are filtered as they climb, so few reach the expensive rungs.

The ladder describes kinds of evidence, and a candidate need not visit every rung: a value-function change may go straight from offline analysis to an A/B test, and a bandit may never face interleaving. Its purpose is to make explicit what each piece of evidence can and cannot establish. Criteo framed off-policy estimation for recommendation as "offline A/B testing" (Gilotte et al., 2018), and Sections 16.7.3 and 16.9.5 describe how Netflix, Airbnb, and Spotify checked the lower rungs against A/B tests.

16.2.3 Experiments, Rollouts, and Release Gates

A staged rollout and an A/B test often share infrastructure, since both route a fraction of traffic to new code, yet they answer different questions with different statistics. Section 15.9.5 drew the boundary: a canary, as described in Section 13.10.3, is tuned to detect harm quickly with a small exposure and accepts statistical imprecision in exchange for speed, while an A/B test is designed for valid inference about the size of an effect. Uber's account of its experimentation platform makes the same distinction, treating staged rollouts, which ramp a feature gradually while continuously monitoring for regressions, as a different activity from standard A/B tests with their own monitoring method (Zhao and Feng, 2017). This chapter is about the second question: whether a change is better, by how much, and for how long.

16.3 Designing an A/B Test for a Recommendation Surface

An A/B test is decided long before it runs. The choices of unit, metric, duration, and decision rule determine what the result can mean, and most of the failures in Section 16.1 were committed at design time.

16.3.1 Hypothesis, Decision Rule, and Pre-Registration

Begin by writing down, before any user is exposed, what the test is for and what result would change a decision. The record should name the hypothesis in plain language ("the multi-task ranker increases weekly completed listens without reducing catalog coverage"), the primary metric that will decide the test, the guardrail metrics that must not degrade, the smallest effect worth detecting, the planned

duration, and the ship rule that maps outcomes to actions. This practice, called pre-registration, protects against reading many metrics after the fact and building a story around whichever one moved.

Netflix describes its A/B tests as decision support and frames the analysis around the decision to be made: which product experience to ship, given the evidence and the costs of being wrong in each direction (Tingley et al., 2021b). Seen this way, a test that finds no effect is still useful, and the decision rule should say in advance what happens in that case. A ship rule might read: launch if the primary metric improves at the agreed significance level and no guardrail shows a significant harm; iterate if the result is inconclusive and the confidence interval still includes effects worth having; abandon otherwise.

16.3.2 Randomization Unit, Analysis Unit, and Assignment

The randomization unit is the entity whose assignment the coin flip decides. Candidates include the user or account, the device or cookie, the session, and the individual request. Recommendation experiments usually randomize users, for three reasons. Personalization state such as embeddings, recent history, and cached candidate lists belongs to the user, so splitting one user's requests across arms mixes the two experiences. Effects carry over between sessions, because a track discovered on Monday changes what the user plays on Tuesday. And a home shelf that reorganizes from one visit to the next is an inconsistent experience that neither arm intends.

The analysis unit is the entity over which a metric is averaged, and it need not match the randomization unit. Click-through rate is randomized by user but naturally computed per impression, a mismatch that changes how variance must be estimated (Section 16.4.3).

Assignment is implemented by hashing. The system computes a hash of the unit identifier concatenated with an experiment-specific salt, maps the result to one of a fixed number of buckets, and assigns buckets to arms. Hashing is deterministic, so a user receives the same arm on every request without a lookup table, and the salt makes assignments independent across experiments.

A recommender team runs many experiments at once: a retrieval change, two rankers, a value-function sweep, and a user-interface test may all be live in the same week. Google's overlapping experiment infrastructure organizes this concurrency with layers, where each layer holds experiments on one set of system parameters and a request is assigned independently in every layer, and domains, which partition traffic so that some experiments can run in isolation (Tang et al., 2010). Experiments in different layers overlap freely, while experiments that change the same parameters, such as two rankers that both alter a slate's final ordering, share a layer and are mutually exclusive. The separate retrieval and ranking services that Section 5.6 recommended let experiments on different stages overlap.

Section 13.6.1 externalized the value function into its own service precisely so that ranking policies can be compared without retraining a model. An experiment arm can therefore be a configuration: a set of value-function weights, a diversity parameter, or a candidate-source quota, selected at request time by the assignment service. Chapter 5 also noted a useful arm that teams often forget, a popularity ranker held as a control group, which measures the incremental value of personalization itself.

16.3.3 Triggering and Dilution

Many users assigned to a ranking experiment never experience a difference. Some never visit the surface being changed. Others visit, but the two rankers happen to produce the same top results for them. Their outcomes are pure noise with respect to the treatment, so including them adds variance while shrinking the measured average effect toward zero, a problem known as dilution.

Trigger analysis restricts the analysis to users whose experience could have differed, called triggered users (Kohavi et al., 2020). The subtle requirement is that the triggering condition must be evaluated identically in both arms. If the system logs a trigger only when a treated user sees a changed list, the control arm has no comparable trigger. The standard remedy is counterfactual logging: for every request in either arm, the system computes what the other arm would have shown and logs whether the two would have differed, so a user is triggered regardless of arm. A trigger decided by something the treatment influences reintroduces the selection bias that randomization removed. Triggered effects also need translating back to the whole population before a launch decision, since a large effect on a small triggered group can be a small effect overall.

16.3.4 Choosing Metrics

Section 2.2.3 introduced a three-tier hierarchy of North Star, proxy, and guardrail metrics for aligning a team with its business. An experiment needs the same hierarchy turned into roles. The overall evaluation criterion, or OEC, is the quantity the experiment is designed to move and the one the ship rule turns on (Kohavi et al., 2020). Ideally it is the North Star itself, but North Star metrics such as retention move slowly and noisily, so the OEC is often a proxy, or a composite of proxies, chosen because it predicts the North Star. Section 16.6.3 returns to how that predictive relationship can be established. Table 16.1 sets out the roles that metrics play around the OEC.

Table 16.1: Metric roles in a recommendation experiment.

Role	What it answers	Recommender examples	Typical analysis unit	Failure if misused
Overall evaluation criterion	Did the change improve what the business ultimately wants?	Retention, long-term engagement, or a composite that predicts them	User	Too slow or noisy to detect realistic effects within the test
Proxy or driver metric	Did user behavior move in the direction expected to lead the North Star?	Click-through, completion, saves, listening time	User or impressi on	Optimizing a proxy that diverges from the North Star
Business guardrail	Did the change harm revenue or the commercial relationship?	Revenue, subscription cancellations, advertiser spend	User or advertise r	Launching a proxy win that erodes revenue
User-experience guardrail	Did the change narrow or degrade what users see?	Diversity, catalog coverage, novelty (Section 8.3)	User or slate	Concentrating exposure while engagement looks healthy
System guardrail	Did the change cost speed or reliability?	Latency, error rate, fallback rate	Request	Attributing to the model an effect

Role	What it answers	Recommender examples	Typical analysis unit	Failure if misused
				caused by slower pages
Trust or data-quality metric	Can the experiment's results be believed at all?	Sample ratio, logging completeness, invariant metrics	Experiment	Reading results without checking that randomization held

Microsoft's experimentation team identified two qualities that a goal metric needs (Deng and Shi, 2016). It must be directional, meaning that a movement in one direction reliably indicates a better experience for users, and sensitive, meaning that it moves detectably when the experience genuinely changes. The two pull against each other. Retention is highly directional but insensitive, while click-through rate is sensitive but, as the opening story showed, its direction can mislead, because clicks can rise when users struggle to find what they want or when a slate turns toward clickbait.

The same team catalogued a dozen recurring ways to misread metric movements (Dmitriev et al., 2017). Three are especially common in recommendation. Ratio metrics can move for the wrong reason: click-through rate rises if a treatment removes low-quality impressions from the denominator, even when total clicks fall, so the numerator and denominator should be reported alongside the ratio. Telemetry loss can be biased: if one variant changes how often client events are lost, for instance by rendering a slower page that users leave before events are sent, the measured difference partly reflects logging. And a metric that showed no significant change in an underpowered test has not been shown to be unaffected; the confidence interval, which may include harmful effects, is the honest summary.

Advertising surfaces need both sides of the marketplace in the metric set. Section 9.3.8 described how advertisers react to return on ad spend and cost per acquisition by changing bids and budgets, so a change that lifts user engagement with sponsored results while worsening advertiser outcomes will not hold its gains.

16.3.5 Trust Checks Before Reading Results

Two checks confirm that an experiment's machinery worked before any effect is read. An A/A test assigns users to two arms that receive identical experiences. Over many A/A tests, or many random re-splits of one, the p-values should be uniformly distributed and roughly five percent should fall below 0.05. A platform whose A/A tests show significant differences much more often has a problem in its assignment, its logging, or its variance estimates, and every A/B result it produces inherits that problem.

The second check is for sample ratio mismatch, or SRM. If the design sends half of users to each arm, the observed counts should be close to equal, with differences explained by chance. A chi-square goodness-of-fit test compares the observed counts with the designed split; in SciPy, `scipy.stats.chisquare` takes the observed and expected counts and returns the test's p-value. A very small p-value means that something other than the coin flip decided which users appear in which arm, so the populations differ in some unknown way and the results cannot be trusted, however attractive they look (Fabijan et al., 2019).

The causes are usually mundane. Fabijan et al. (2019) organize them into five types: four by the stage at which they arise (assignment, execution, log processing, and analysis) and a fifth for interference by

experimenters, such as ramping one arm unevenly or pausing it. One of their examples comes from a carousel experiment at Microsoft, where the treatment's most engaged users crossed the activity threshold of a bot detector and were silently removed from the analysis. Recommenders offer hypothetical variants of their own: if a new heavy ranker times out more often, the orchestrator may fall back to a default path that logs requests differently, so slow sessions vanish from the treatment arm's logs. As Chapter 4 warned, any difference in how two paths produce or process logs can surface here as a sample ratio mismatch.

Invariant metrics provide a third check. These are quantities that the treatment cannot plausibly affect, such as the distribution of users by country or by account age at assignment. A significant difference in an invariant metric signals that the arms are not comparable.

Companion code. Run `ab_test_simulation.py --mode aa` to split real Yambda users repeatedly into identical arms and check that the p-values are uniform, then `--mode srm`, which injects a bug that silently drops some heavy users from one arm. Compare the naive metric difference, which looks like a convincing treatment effect, with the chi-square test, which flags the broken split in most runs. Add `--synthetic` to either command if the Yambda data is not available locally.

16.4 Power, Sensitivity, and Variance Reduction

A well-designed test can still fail to see an effect that exists, or announce one that does not. This section is about controlling both errors when effects are small and engagement data is noisy, which describes nearly every ranking experiment.

16.4.1 A Compact Statistical Refresher

An experiment tests a null hypothesis, usually that the treatment has no effect ($\tau = 0$). The p-value is the probability, computed assuming the null hypothesis is true, of observing a difference at least as extreme as the one observed. A test rejects the null when the p-value falls below a significance level α, conventionally 0.05. A 95 percent confidence interval is the range of effect sizes consistent with the data at that level, and it is usually more informative than the p-value, because it shows both the direction and the plausible size of the effect.

Two errors are possible. A Type I error, or false positive, declares an effect when none exists, and α is its intended rate. A Type II error, or false negative, misses an effect that exists, and its rate is written β. Power is $1 - \beta$, the probability of detecting an effect of a given size, and 80 percent is the common convention. Netflix's introduction to power describes it as the protection against false negatives and explains what determines it: the size of the effect, the number of units, and the variability of the metric (Tingley et al., 2021a).

The minimum detectable effect (MDE), written δ, is the smallest true effect the test is designed to detect with the chosen power. For a two-arm test comparing means with equal allocation, the number of units required per arm is approximately

$$n \approx \frac{2(z_{1-\alpha/2} + z_{1-\beta})^2 \sigma^2}{\delta^2},$$

where σ^2 is the variance of the metric per unit and z_q is the q quantile of the standard normal distribution. With $\alpha = 0.05$ and 80 percent power, the numerator constant is close to 16, which gives the rule of thumb $n \approx 16\sigma^2/\delta^2$ (Kohavi et al., 2020). Halving the detectable effect quadruples the required sample; halving the variance halves it.

16.4.2 Why Recommendation Metrics Are Hard to Move

Two properties of recommendation data push n upward. The first is heavy-tailed engagement. A small fraction of users listen, watch, or shop far more than everyone else, so per-user metrics such as weekly listens have a long right tail and a variance dominated by those few heavy users. A handful of them landing in one arm can shift the arm's mean more than any realistic treatment would.

The second is that ranking changes usually have small effects. A mature recommender has already captured most of the easy gains, and a new ranker reorders items within a slate that was already reasonable. The MDE that matters for the business is therefore small relative to σ, and the formula above punishes small ratios of δ to σ quadratically.

The simplest defence against the heavy tail is capping, also called winsorizing: replacing each user's value above a high percentile with the value at that percentile. Capping can reduce variance substantially at the cost of a small bias in what is being measured. The threshold must be chosen before the test, ideally from pre-experiment data, because a cap chosen after seeing results is another way of searching for significance (Kohavi et al., 2020).

16.4.3 Ratio Metrics and the Delta Method

Many recommendation metrics are ratios: click-through rate is clicks divided by impressions, completion rate is completed listens divided by started listens. The experiment randomizes users, yet the ratio is naturally computed per impression, so a natural but wrong analysis treats every impression as an independent observation and computes a standard error for a proportion. Impressions from the same user are correlated, because a user who clicks a lot does so across many impressions. Treating them as independent understates the variance and inflates the false positive rate. Uber reported the same effect in its rollout monitoring, where sessions from the same user were highly correlated and a test that assumed session independence produced far too many false alarms (Zhao and Feng, 2017).

The correct analysis treats the user as the independent unit. Let user i contribute S_i clicks and N_i impressions, and let the metric be the ratio of means $R = \bar{S}/\bar{N}$ over the n users in an arm. The delta method, a first-order Taylor approximation, gives

$$\operatorname{Var}(R) \approx \frac{1}{n\mu_N^2}\left(\sigma_S^2 - 2R\sigma_{SN} + R^2\sigma_N^2\right),$$

where μ_N is the mean number of impressions per user, σ_S^2 and σ_N^2 are the per-user variances of clicks and impressions, and σ_{SN} is their covariance. Every quantity is computed from per-user totals, so the formula respects the randomization unit while still reporting the per-impression ratio. Deng et al. (2018) give a practical treatment of this and related cases. A user-level bootstrap, which resamples whole users with replacement and recomputes the ratio each time, provides a slower check that needs no formula.

16.4.4 Variance Reduction

Much of the variance in an experiment metric reflects stable differences between users that existed before the test began. A user who listened heavily last month will probably listen heavily this month in either arm. Removing that predictable component leaves a much smaller residual variance in which a treatment effect is easier to see.

CUPED (controlled experiments using pre-experiment data) does this with a single covariate X_i, usually the same metric measured for the same user during a period before the experiment (Deng et al., 2013). The adjusted metric is

$$Y_i' = Y_i - \theta(X_i - \bar{X}), \theta = \frac{\text{cov}(X, Y)}{\text{var}(X)},$$

and the treatment effect is estimated as the difference in mean Y' between arms. If ρ is the correlation between X and Y, the variance of the adjusted metric is $(1-\rho^2)$ times the original, so a covariate that correlates well with the outcome reduces the required sample size by the same factor. The estimate stays unbiased because X was measured before assignment and therefore cannot be affected by the treatment; randomization makes its mean equal in expectation across arms. The same reasoning rules out covariates measured during the experiment, which the treatment may have changed.

Figure 16.2 shows both halves of the idea.

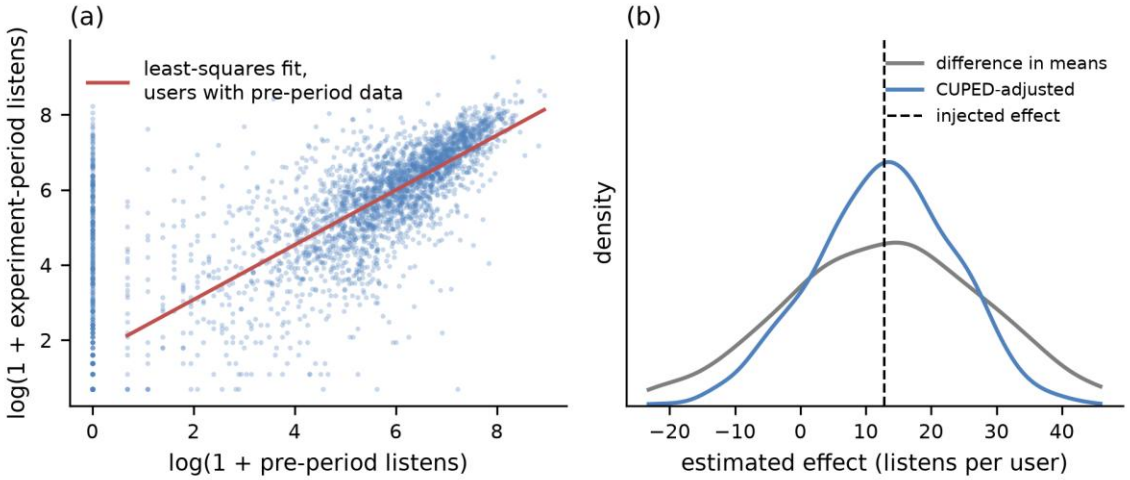


Figure 16.2: Variance reduction with pre-period data (CUPED), on real listening histories with a simulated treatment effect. (a) Each user's listens before the experiment predict that user's listens during it, on log(1 + listens) axes; users with no pre-period listens sit at zero, and the fit uses the others. (b) Across 1,000 simulated experiments with a 2 percent injected effect, about 13 listens per user, the raw difference in means and the CUPED estimate are both centred near the injected effect, and the CUPED estimate's variance is about half as large.

Booking.com described its adoption of CUPED in 2018, using the same metric from the pre-experiment period as the covariate where possible and leaving the metric unadjusted for units with no pre-experiment data, which keeps those units in the analysis (Jackson, 2018). Netflix compared stratified sampling, which balances arms on user characteristics at assignment time, with post-stratification and CUPED, which adjust for the same characteristics after assignment. Its case studies found comparable variance reduction from the post-assignment methods, which also avoid the difficulties it described in stratifying users in real time, and it recommended them for large-scale experiments (Xie and Aurisset, 2016). DoorDash generalized the covariate to a machine-learned prediction of the outcome built only from features the treatment cannot affect, a method it named CUPAC (control using predictions as covariates). The variance removed grows with how well the prediction correlates with the outcome, so the model is tuned for that correlation, and DoorDash applied it to the switchback experiments that Section 16.5 describes (Li, 2020).

16.4.5 Peeking and Sequential Testing

Return to the dashboard from Section 16.1. A fixed-horizon test computes its p-value once, at a sample size chosen in advance, and only then is the false positive rate equal to α. Under the null hypothesis the running test statistic wanders, so checking it every day and stopping at the first significant reading gives

chance many opportunities to cross the threshold. Johari et al. (2017) showed that this habit, which they call peeking, inflates false positive rates far beyond their nominal level in exactly the way that real experimenters use dashboards. Figure 16.3 shows the effect on A/A tests of real listening data.

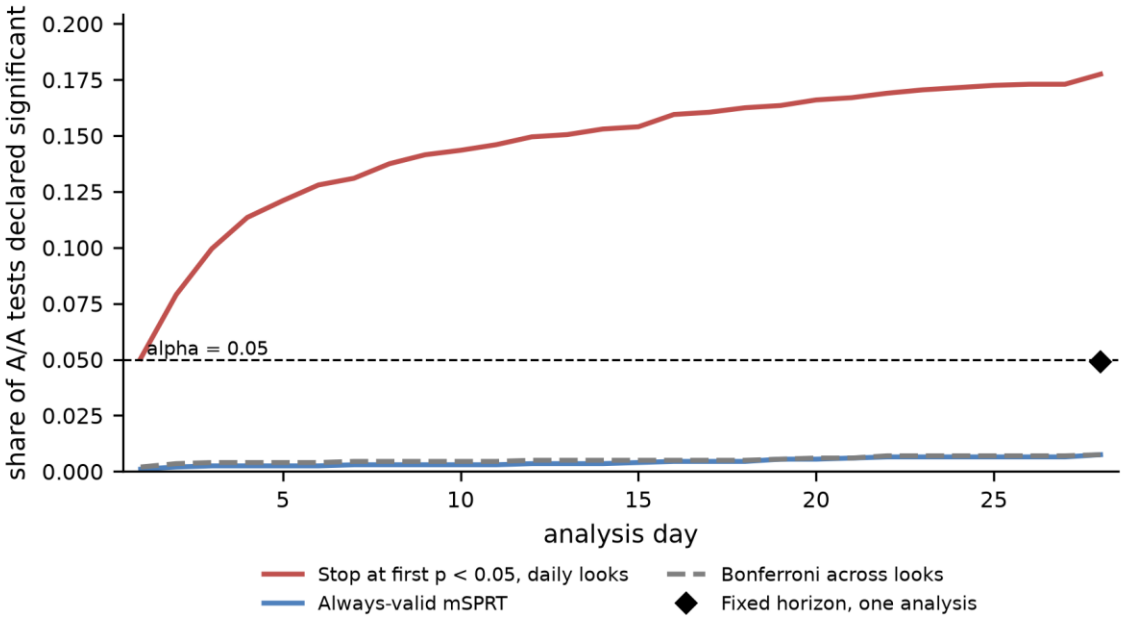


Figure 16.3: Peeking inflates false positives. In 2,000 A/A splits of real listening data, checked once a day for 28 days, the share declared significant at some point keeps rising, to about 18 percent, when the team stops at the first p-value below 0.05. A Bonferroni correction across looks and an always-valid mixture sequential probability ratio test stay well below the nominal 5 percent, and a single analysis at the planned end sits at it.

Sequential tests allow repeated looks without that inflation, and two families dominate. Group sequential tests plan a fixed number of interim analyses and spend the overall α across them according to a spending function, using stricter thresholds early and looser ones late. Always-valid tests, such as the mixture sequential probability ratio test (mSPRT) that Johari et al. (2017) developed and deployed at Optimizely, produce p-values that remain valid however often they are checked and whenever the test stops. The flexibility has a price in power: for the same data, a test that permits unlimited looks is less likely to detect a real effect than one that plans its looks.

Spotify described in 2023 how it chose between these families (Schultzberg and Ankargren, 2023). Its comparison turned on two questions: does the data arrive in batches or as a stream, and can the maximum sample size of an experiment be estimated in advance? Spotify's data mostly arrived in daily batches and its tests ran for weeks with predictable traffic, so it used group sequential tests, which had the highest power in its simulations when the sample size could be estimated. Always-valid tests suit streaming data and open-ended tests. When the number of looks is small, even a simple Bonferroni correction, which divides α by the number of looks, was competitive.

16.4.6 Many Metrics and Many Variants

The arithmetic that punishes peeking also punishes reading many metrics or variants: twenty independent metrics at $\alpha = 0.05$ yield about one false positive even when nothing changed. The first defence is a single primary metric, named in the pre-registration. When several comparisons must be made, the Bonferroni correction tests each of m comparisons at α/m, controlling the probability of any

false positive at a conservative price. The Benjamini-Hochberg procedure controls the false discovery rate, the expected share of declared effects that are false, by comparing sorted p-values with a sliding threshold; it suits exploratory reads of secondary metrics.

Guardrails are better tested one-sided, for harm only. Netflix also emphasizes reporting every experiment, including those with no significant effect, because at a 5 percent false positive rate some significant results are false positives, and only a full record shows how many tests were run (Tingley et al., 2021b).

The value-function weight sweeps of Section 8.2.3 are the canonical multi-variant case. A sweep of several configurations against one control, the default tuning strategy in Figure 8.2, faces multiple comparisons and a sample divided among more arms, and each arm spends user exposure from the error budget of Section 15.2.2. A sweep with many arms should be planned with its correction, per-arm power, and budget written down, or replaced by the adaptive allocation of Section 16.8.

Companion code. `ab_test_simulation.py` runs on semi-synthetic data by default: real Yambda users and their listening history, hashed arm assignment, and effects of known size injected into the treatment arm. With `--synthetic` it uses simulated users instead. `--mode power` compares power curves for raw, capped, and CUPED analyses; `--mode cuped` reproduces Figure 16.2; `--mode ratio` compares the coverage of naive, delta-method, and bootstrap confidence intervals in A/A runs; and `--mode peeking` reproduces Figure 16.3.

16.5 Interference: When Treatment Leaks

Everything so far has assumed the first half of SUTVA from Section 16.2.1: each user's outcome depends only on that user's own assignment. A user-randomized test then compares a world where everyone is treated with a world where no one is, using two halves of the same population. When treatment leaks between users, the two halves no longer represent those two worlds, and the difference in their means estimates something else. Recommenders leak in at least four ways.

16.5.1 Four Channels of Interference in Recommenders

The first channel is shared training data. A production recommender usually retrains on logs from all of its traffic, so during a test the model serving control users learns partly from what the treatment showed, and the reverse. Suppose the treatment is a ranker that promotes newly released tracks. Its users generate engagement data on those tracks, the shared model learns that they are worth showing, and control users begin to see them too. The measured difference between arms shrinks, though the launched treatment would have changed the training data for everyone. Brennan et al. (2025) name this effect symbiosis bias, since each algorithm's performance is shaped by data that its competitor generated, and they confirmed it in the field with data from a large-scale A/B test on a global recommender. Si (2023) analyses the same data-training loop and shows in simulation how it biases A/B estimates.

The second channel is shared inventory or budgets. In a marketplace, the treatment and control arms compete for the same supply. If a new ranker sends more users toward a popular listing, a hotel room, or a delivery slot, treatment users may book items that control users would otherwise have booked, so the treatment's apparent gain is partly a transfer from control (Johari et al., 2022). Advertising adds a sharper version, because sponsored results draw on finite advertiser budgets. Section 9.3.8 described how advertisers react to auction outcomes; within a single experiment, a treatment that wins more

auctions can exhaust an advertiser's daily budget earlier, leaving less to spend on control traffic and inflating the measured lift (Liu et al., 2021).

The third channel is social connection. When users share, message, or watch content together, a treated user's recommendations reach untreated friends. Experiments on social products therefore measure a mixture of direct and indirect effects, and the indirect part leaks into the control group (Ugander et al., 2013; Saint-Jacques et al., 2019).

The fourth channel is the supply side. Creators, sellers, and publishers respond to the exposure that the recommender gives them. A policy that gives fresh content more exposure may encourage creators to upload more, enlarging the corpus that every user, in either arm, can be shown. A user-randomized test cannot see this effect at all, because the corpus is shared. Su et al. (2024) reported that ordinary user-randomized A/B tests of exploration on a large short-form video platform often measured neutral or negative engagement effects, while experiment designs that captured the effect on the content corpus revealed its long-term value.

16.5.2 Designs That Restore Validity

Each channel has designs that contain it, and each design changes what is randomized so that interference stays within a randomized block. Figure 16.4 shows the assignment structure of four of them, and Table 16.2 summarizes their trade-offs.

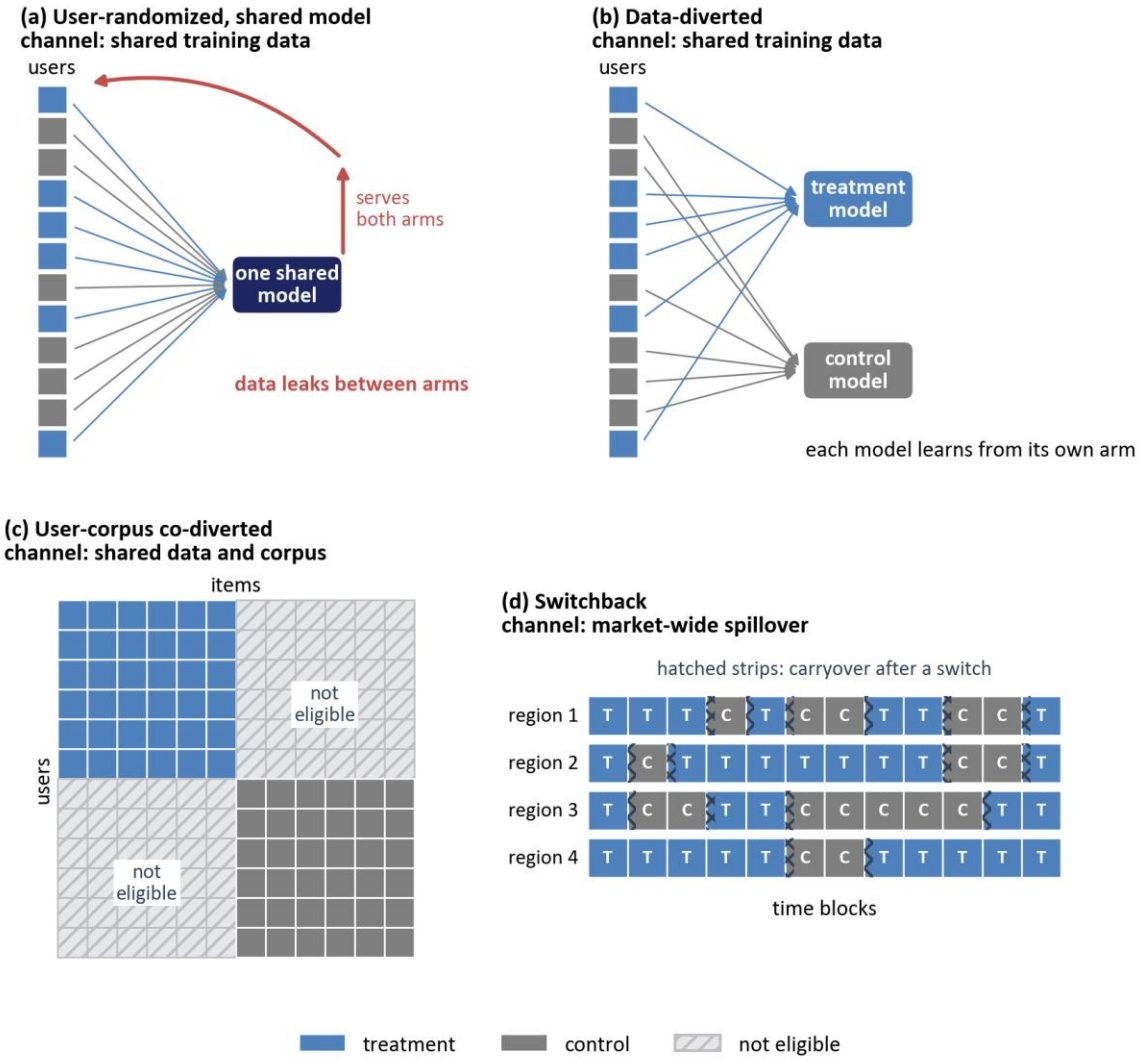


Figure 16.4: How experiment designs contain interference. (a) In a user-randomized test with a shared model, logs from both arms train one model that serves both, so data leaks between arms. (b) A data-diverted design trains a separate model for each arm on that arm's logs only. (c) A user-corpus co-diverted design splits items as well as users, so treatment users see only treatment items and the cross-arm combinations are ineligible. (d) A switchback design assigns whole regions to treatment or control for blocks of time, with carryover near each switch.

Cluster randomization assigns groups of connected units together, so that most of a user's connections share that user's arm. Graph cluster randomization partitions the social graph into densely connected clusters and randomizes whole clusters (Ugander et al., 2013). LinkedIn's ego-cluster design takes each focal user, the ego, and randomizes that user's connections, the alters, as a block, which lets the experiment compare egos whose networks were treated with egos whose networks were not (Saint-Jacques et al., 2019).

Marketplaces can randomize the other side. Listing-side randomization assigns items to arms and applies the treatment to how treated items are ranked for every user, while two-sided randomization assigns both users and items and applies the treatment only where a treated user meets a treated item. Johari et al. (2022) showed in a marketplace model that which side to randomize depends on market balance: customer-side randomization is unbiased when demand is scarce, listing-side randomization is

unbiased when supply is scarce, and for positive interventions both tend to overstate the effect away from those extremes. No single side is safe by default.

For advertising, the budget-split design divides each advertiser's budget into separate shares for the two arms, with users randomized as usual, so each arm's auctions can only spend that arm's share and treatment cannot cannibalize control's budget. LinkedIn developed and deployed this design for its ad marketplace (Liu et al., 2021).

Switchback designs randomize over time instead of over users. A region, such as a city, alternates between treatment and control in blocks of time, and the analysis compares periods. This contains market-wide interference, since at any moment the whole market is in one arm, but it introduces carryover: effects from one period can persist into the next, for example when deliveries assigned under treatment complete under control. Bojinov et al. (2023) analyse how to choose the switch interval and handle carryover, and Section 16.4.4 noted that DoorDash reported applying CUPAC to its switchback tests.

The shared-training-data channel needs designs that separate the data. In a data-diverted design, each arm's model trains only on logs from that arm's users, so neither learns from the other. In a user-corpus co-diverted design, items are also split in proportion to users, so the treatment's effect on the corpus, such as more fresh uploads or more tail items becoming discoverable, stays within the treatment arm and can be measured there (Su et al., 2024; Brennan et al., 2025). An analysis-side alternative, weighted training, reweights the logs used to train each arm's model to reduce the influence of the other arm's data; it has been proposed and studied in simulation (Si, 2023).

Table 16.2: Experiment designs under interference.

Design	Interference addressed	What is randomized	Main cost	Recommender setting	Source
User-randomized (baseline)	None	Users	Biased under any channel	Default for interface and ranking changes	Kohavi et al. (2020)
Cluster or ego-cluster	Social spillover	Graph clusters, or egos with their alters	Fewer units, more variance	Social feeds and sharing	Ugander et al. (2013); Saint-Jacques et al. (2019)
Listing-side or two-sided	Competition for shared inventory	Items, or users and items	Complex analysis; bias depends on market balance	Marketplaces and lodging	Johari et al. (2022)
Budget-split	Advertiser budget competition	Budget shares per arm, plus users	Needs budget-splitting infrastructure	Ad marketplaces	Liu et al. (2021)

Design	Interference addressed	What is randomized	Main cost	Recommender setting	Source
Switchback	Market-wide or temporal spillover	Time blocks within regions	Carryover; few effective units	Marketplaces and delivery	Bojinov et al. (2023)
Data-diverted	Shared training data	Users, with per-arm training data	Less training data per model	Model changes under continuous training	Brennan et al. (2025)
User-corpus co-diverted	Shared training data and corpus or creator effects	Users and items in proportion	Smaller eligible corpus per arm	Exploration and fresh-content policies	Su et al. (2024); Brennan et al. (2025)

16.5.3 Choosing a Design

Every design in Table 16.2 removes one bias by paying somewhere else. A data-diverted design gives each arm's model only part of the training data, so a treatment arm holding a tenth of traffic trains on a tenth of the logs, and the test may measure the effect of less data as much as the effect of the new algorithm. A co-diverted design shrinks the corpus that each arm's users can be shown, which changes the experience being tested. Cluster and switchback designs reduce the number of effective units, from millions of users to thousands of clusters or a few dozen region-periods, and variance rises accordingly. Each design also adds engineering, from partitioned training pipelines to budget accounting.

The right design depends on which channel dominates. A user-interface change that neither alters training data nor competes for scarce inventory is well served by a user-randomized test. A new ranker under continuous training is exposed to the shared-data channel, an exploration policy to the corpus channel, and an ads ranking change to the budget channel. When it is unclear whether a channel matters, the bias can itself be measured: running a data-diverted test alongside a standard one for the same change shows how far the two estimates differ.

Sometimes the direction of the bias can be reasoned out even when its size cannot, and then a user-randomized estimate is still useful as a bound. If the treatment generates data or content that also benefits control users, as with exploration, the measured difference is likely to understate the launched effect. If the treatment wins by drawing scarce inventory or budget away from control, the measured difference is likely to overstate it. Either way, the reasoning should be recorded with the result; these are the feedback mechanisms of Section 15.2.3, acting within the experiment.

16.6 Effects That Change Over Time

The average treatment effect of Section 16.2.1 was written as a single number, yet the effect of a recommendation change is rarely constant over time. Users notice changes, react to them, adapt, and gradually learn what the new system offers. A two-week test measures the effect during those two weeks, and a launch decision is a bet on all the weeks that follow.

16.6.1 Novelty and Primacy

A novelty effect is a treatment effect that starts high and decays as the change stops being new. A redesigned shelf or a noticeably different mix of tracks draws curiosity clicks from users exploring what changed, and those clicks fade once the change becomes familiar. A primacy effect is the reverse: users accustomed to the old experience at first engage less with the new one, because their habits no longer fit it, and engagement rises as they adapt. Section 2.5.2 advised running experiments long enough for novelty to fade. That advice is sound but incomplete, since it does not say how long is long enough or how to tell whether fading has happened.

Detection starts with plotting the effect against time since each user's first exposure, as in Figure 16.5. Users enter an experiment on different days, when they first visit the affected surface, so day zero of exposure falls on different calendar days for different users. Aligning users by days since first exposure separates the effect of familiarity from calendar events such as weekends and holidays. A downward slope suggests novelty, an upward slope suggests primacy, and a flat line suggests that the pooled estimate can be trusted. A second check compares users who were new to the product during the test with established users, since new users have no old habits to unlearn and show little primacy.

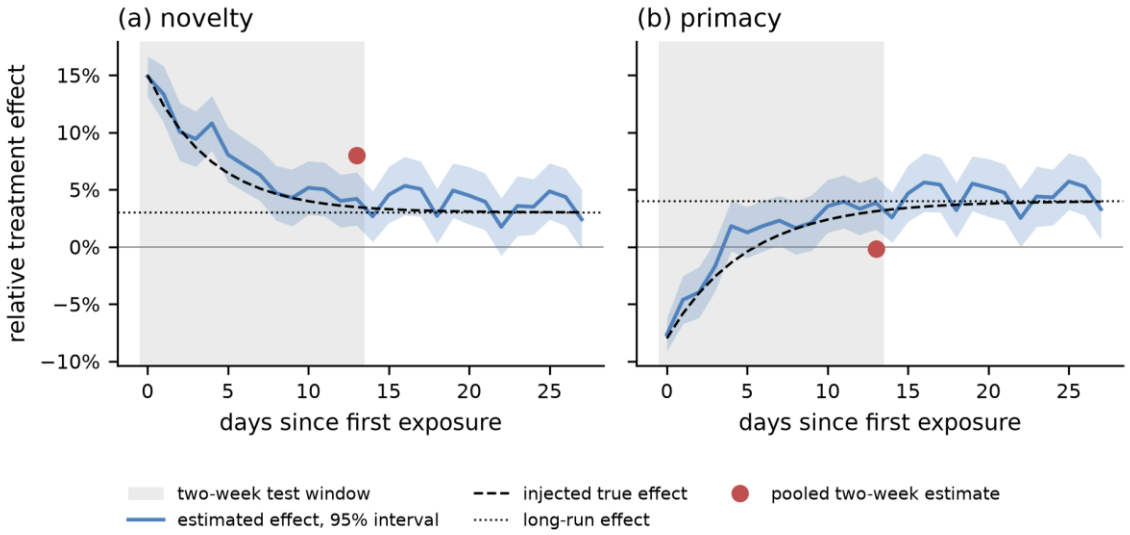


Figure 16.5: Novelty and primacy effects over time, on real listening histories with simulated effects. Each panel plots the estimated daily treatment effect, with a 95 percent confidence band, against days since a user's first exposure, with the injected effect curve and its long-run level. A shaded band marks a two-week test window, and a marker shows the pooled estimate from that window. (a) Under novelty, the effect decays from 15 percent, so the pooled estimate, about 8 percent, overstates the lasting 3 percent. (b) Under primacy, the effect starts negative and rises, so the pooled estimate, near zero, understates the lasting 4 percent.

Microsoft developed an estimator that separates user learning, its term for novelty and primacy together, from the long-run effect (Sadeghi et al., 2022). The approach uses difference-in-differences, a technique that compares how the gap between treatment and control changes across groups of users who were first exposed at different times, and Microsoft applied it to many of its experiments.

16.6.2 Long-Term Effects and User Learning

Some effects take months to reveal themselves. Google studied how the quantity and quality of search ads affected users over long periods and found that users learned: sustained exposure to more or worse

ads changed how users responded to ads over time, so short experiments mis-estimated long-term effects (Hohnhold et al., 2015). Two designs from that work show how learning can be measured. A post-period analysis keeps the treatment running for an extended time, then returns both arms to the same experience and compares them, so that any remaining difference reflects what treated users learned. A cookie-cookie-day design treats a fresh set of users each day, which allows the team to compare users with different lengths of exposure on the same calendar day. Although the setting was advertising, the lesson carries over to recommendation: a ranker that shows users fewer surprises may train them to expect less from the product.

The most direct tool for long-term measurement is the long-term holdback, which keeps a small, randomly chosen group of users on the old experience after a change launches to everyone else, and compares the two groups for months (Kohavi et al., 2020). This is the mechanism that exposed the problem in Section 16.1. The term needs separating from two similar ones in earlier chapters. Chapter 3's held-out evaluation set, often called a holdout set, is data withheld from training for offline evaluation. Chapter 9's advertiser holdout experiments are incrementality tests that advertisers run on their own campaigns. A holdback is a group of users withheld from a launched change, and it is an online experiment. Holdbacks have costs, since the held-back users receive the presumably worse experience for months, and they degrade over time as users churn and as the product changes around them.

Duration matters even for ordinary tests. Two forces set its minimum. Label maturation, discussed in Section 15.7 and, for conversions, in Section 9.6.1, means that outcomes such as completions, purchases, or renewals arrive after the exposure that caused them, so a test must run long enough for its metrics to mature. Weekly seasonality means that weekend and weekday users behave differently, so tests should run for whole weeks.

16.6.3 Surrogates and Learned Proxies

Holdbacks answer the long-term question slowly, so teams try to predict long-term effects from short-term ones, as Chapter 2's proxy tier does informally. A surrogate is the formal version of a proxy: a short-term outcome assumed to carry the treatment's entire effect on the long-term outcome. Athey et al. (2026) combine several short-term outcomes into a surrogate index, the predicted long-term outcome learned from historical data where both were observed, and show that the effect on the index equals the long-term effect when the long-term outcome is independent of the treatment given the surrogates. Their application was a job-training programme; in a recommender, a treatment that affects retention through a path the surrogates miss breaks the assumption.

Netflix learned how effects on short-term proxies relate to effects on long-term outcomes from its archive of past experiments (Bibaut et al., 2024). Its researchers warned that fitting a line through estimated proxy effects against estimated North Star effects is misleading, because both estimates share measurement noise from the same users, which biases the slope; the true relationship can even be negative while the naive slope is positive. A proxy earns its role through evidence about treatment effects, which user-level correlation alone cannot supply.

Companion code. `ab_test_simulation.py --mode novelty` reproduces Figure 16.5 from resampled Yambda users, or uses simulated users with `--synthetic`. It enters users on their first active day and injects an effect that changes with days since entry, in a novelty scenario and a primacy scenario. Compare the daily estimates with the true curve and the pooled two-week estimate to see how a short test misleads in each direction.

16.7 Interleaving for Ranking Comparisons

Section 2.5.2 promised interleaving as an alternative to traditional A/B tests. The method is specific to ranking and trades breadth of measurement for a large gain in sensitivity.

16.7.1 The Paired-Comparison Idea

An A/B test compares two groups of users, so its precision is limited by how much users differ from one another, and Section 16.4.2 showed how large those differences are in engagement data. Interleaving removes that source of variance by showing each user a single list that mixes results from both rankers, then crediting each click to the ranker that contributed the clicked item. Every user becomes a comparison between the two rankers, and heavy and light users alike contribute a single preference.

The idea comes from web search. Radlinski et al. (2008) ran an experiment on the arXiv.org full-text search engine in which they deliberately degraded the ranking function to create pairs of rankers with known quality differences. Absolute usage metrics, such as clicks per query and time to first click, often failed to reflect those known differences reliably, while paired comparisons by interleaving identified the better ranker consistently. Chapelle et al. (2012) later validated interleaving at the scale of commercial web search, comparing its verdicts with other evaluation methods and analysing its sensitivity.

16.7.2 Mechanics

The earliest scheme, balanced interleaving, builds the merged list by alternately taking items from the two rankings so that, at every depth, the merged list contains roughly equal-length prefixes of both. To credit clicks, the method finds the smallest depth at which the lowest clicked result appears in either original ranking, then counts how many clicked results each ranking holds within that depth. Radlinski et al. (2008) showed that this scheme can favour one ranker when the two rankings are nearly identical, for example when one ranking is the other shifted down by a single position, and introduced team-draft interleaving to remove that bias.

Team-draft interleaving borrows from how children pick teams in a schoolyard game. For each request:

1. A coin flip decides which ranker picks first in the current round.
2. Each ranker in turn adds its highest-ranked item that is not already in the merged list, and the item is tagged as belonging to that ranker's team.
3. Rounds repeat until the merged list reaches the desired length.
4. When the user interacts with the list, each click is credited to the team of the clicked item.
5. The impression counts as a win for the ranker whose team earned more clicks, a loss for the other, or a tie if they earned equal clicks or none.

Across many impressions, the wins are aggregated. Under the null hypothesis that users have no preference, a non-tied impression is equally likely to be a win for either ranker, so the number of wins among non-tied impressions follows a binomial distribution with probability one half. A binomial test, which in SciPy is `scipy.stats.binomtest`, gives the p-value. Figure 16.6 traces one impression through these steps.

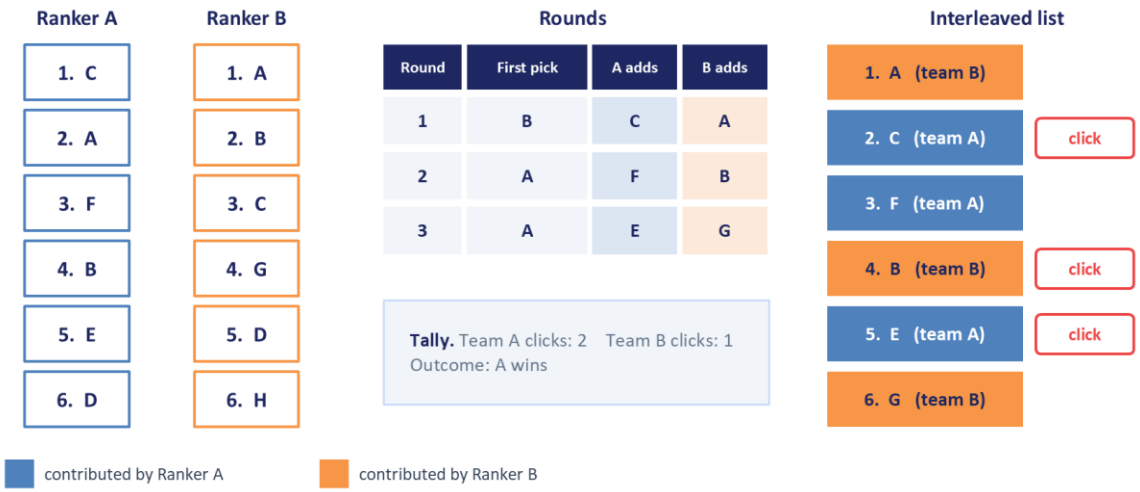


Figure 16.6: Team-draft interleaving for one impression. Two rankers order overlapping sets of candidate items differently. In each round a coin flip decides which ranker picks first, and each adds its highest-ranked item not yet chosen, tagged with its team. The user's clicks on the interleaved list are credited to the teams that contributed the clicked items, and the impression is scored as a win, loss, or tie.

The coin flip in every round is what makes the comparison fair. Users click more on higher positions regardless of relevance, and because each ranker is equally likely to fill any given position, position bias affects both teams equally in expectation.

Production systems have adapted the method to recommendation layouts. Netflix described in 2017 a variant of team-draft interleaving for rows on its homepage: the two rankers alternately contribute videos to a single row, and the preferred ranker is the one whose contributed videos account for the greater share of hours viewed, with viewing attributed to the ranker that supplied each video (Parks et al., 2017). Airbnb described in 2022 a team-draft variant for search ranking built on competitive pairs. When the two rankers' next choices differ, the pair is added to the merged list with the order set by the coin flip and each item credited to its team; when they choose the same listing, it is added without credit to either. Shared items therefore contribute no noise. Because Airbnb's goal is bookings, it credited bookings instead of clicks and designed attribution rules for guests who see the same listing from both teams across several searches (Zhang et al., 2022).

16.7.3 Sensitivity and the Two-Stage Funnel

Because interleaving compares rankers within each user, it typically needs far fewer users than an A/B test to reach a correct decision about which ranker is better. Netflix reported that interleaving identified the preferred ranker with a small fraction of the users that an A/B test needed, and that its interleaving preference agreed closely with its most sensitive A/B evaluation metric (Parks et al., 2017). Airbnb reported a large gain in sensitivity and high agreement with A/B tests for search ranking (Zhang et al., 2022). The sizes of these gains belong to each company's surfaces and metrics, and Figure 16.7 shows the same qualitative pattern in a simulation.

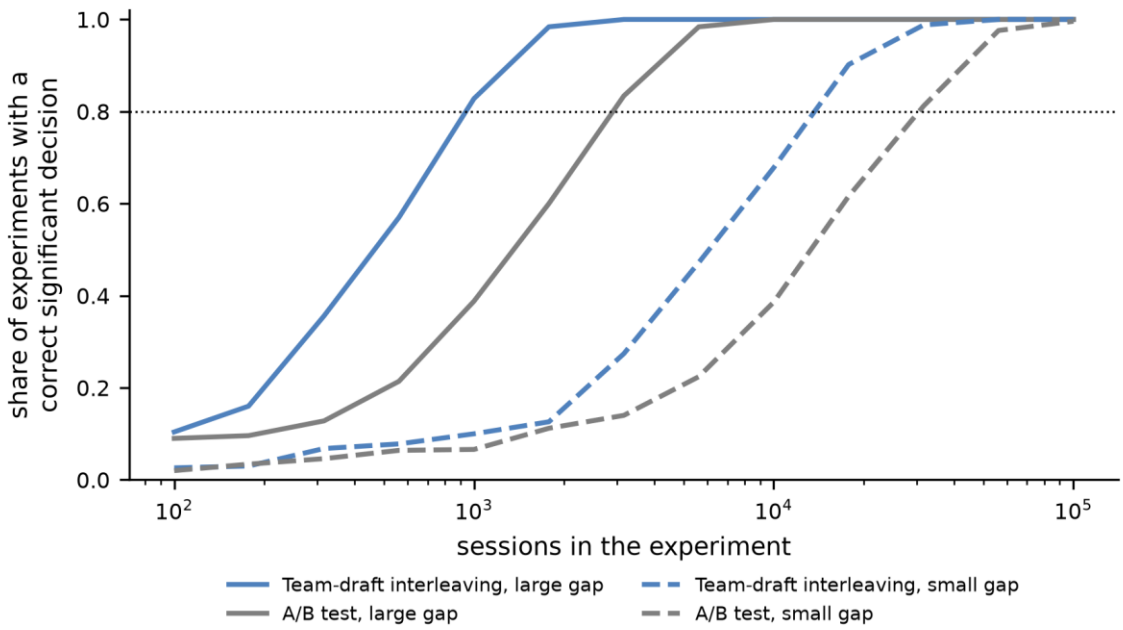


Figure 16.7: Interleaving reaches correct decisions with fewer sessions, simulated. For two pairs of rankers, one with a small quality gap and one with a large gap, the curves show the fraction of simulated experiments that reach a significant result in the correct direction as the number of sessions grows. Team-draft interleaving reaches high power with a fraction of the sessions that an A/B test on clicks per session needs.

Both companies use this sensitivity in a funnel: interleaving prunes a large set of candidate rankers quickly, and A/B tests then measure the survivors' effect on business outcomes.

16.7.4 Limits

Interleaving measures preference and leaves magnitude unmeasured: it says that users prefer ranker B, without saying how much retention or revenue would change if B replaced A. Because each user sees one blended list, effects on slate diversity, session length, long-term retention, and latency are invisible. Airbnb found that most disagreements between interleaving and A/B tests arose from set-level optimization, where a ranker's choices depend on the rest of the page, an assumption that a blended list breaks (Zhang et al., 2022). The same caution applies to the value-function reranking of Chapter 8, whose diversity terms are properties of a whole slate.

There are practical costs as well. Both rankers must run for every interleaved request, which roughly doubles the ranking work for that traffic and counts against the latency budget discussed in Chapter 14. Grids and carousels complicate credit assignment, because position effects in two dimensions are harder to balance than in one list. And when two rankers rarely differ near the top of the list, most impressions are ties and the test learns little; this is informative in its own right, since a candidate that changes so little is unlikely to move business metrics either.

Companion code. Run `interleaving_simulation.py --mode sanity` to confirm that identical rankers produce balanced wins and the nominal false positive rate. `--mode sensitivity` reproduces Figure 16.7, comparing team-draft interleaving with an A/B test on simulated sessions with a position-based click model, and `--mode example` traces one seeded impression step by step, in the format of Figure 16.6. All three modes use simulated data.

16.8 Bandits: Learning While Serving

Chapter 1 named contextual bandits, Thompson sampling, and epsilon-greedy as tools covered here, Section 3.4.2 promised explore-exploit strategies for new items, and Figure 8.2 listed bandits among the ways to tune value-function weights. An A/B test holds allocation fixed and learns at the end, while a bandit learns during serving and shifts traffic toward what works as evidence accumulates.

16.8.1 Why a Recommender Must Explore

Section 3.4.1 described the feedback loop at the centre of every recommender: the system collects interactions only on the items it shows, trains on them, and shows more of what it already favours. An item that the model underrates is rarely shown, so the evidence that would correct the model never arrives. Jiang et al. (2019) analysed these degenerate feedback loops formally and showed that a system which always shows its current best guesses tends toward degeneracy, with user interests narrowing to what the system already recommends, and that exploration, along with a growing candidate pool, slows that degeneracy.

Exploration means occasionally choosing an action that the current model does not rank first, in order to learn about it. Exploitation means choosing the action with the highest estimated reward. The exploration-exploitation trade-off is the tension between them: exploration costs reward now and buys information that can earn more reward later. In a bandit problem, a learner chooses one of several actions, called arms, in each round t, observes a reward r_t for the chosen arm only, and updates its estimates. Its performance is measured by regret, the cumulative reward lost relative to always choosing the best arm:

$$\text{Regret}(T) = \sum_{t=1}^{T} \left(\mu^* - \mu_{a_t}\right)$$

where μ_a is the expected reward of arm a, μ^* is the expected reward of the best arm, and a_t is the arm chosen in round t. A good algorithm has regret that grows much more slowly than the number of rounds, so its average loss per round shrinks toward zero.

16.8.2 Multi-Armed Bandits

The simplest policy is epsilon-greedy: with probability ϵ, choose an arm uniformly at random, and otherwise choose the arm with the highest observed mean reward. It is easy to implement and to explain, and its exploration probabilities are known exactly, which matters for Section 16.9. Its weakness is that it explores blindly, spending as much on arms that are clearly bad as on arms that are nearly tied with the leader.

Upper confidence bound algorithms explore more selectively, following a principle called optimism in the face of uncertainty. UCB1 chooses the arm with the highest index

$$\bar{x}_a + \sqrt{\frac{2\ln t}{n_a}}$$

where $\bar{x}_a$ is the observed mean reward of arm a and n_a is the number of times it has been chosen. The second term is a bonus that is large for rarely tried arms and shrinks as an arm accumulates trials. An arm is chosen either because its mean is high or because so little is known about it that it could plausibly be the best. Auer et al. (2002) proved that UCB1's regret grows only logarithmically with the number of

rounds. The bound holds at every horizon, but its constant grows as the gaps between the arms' reward rates shrink, and the bonus is sized for rewards anywhere between zero and one. When reward rates are low and close together, UCB1 therefore keeps exploring for a long time and can trail a simple epsilon-greedy policy, as Figure 16.8(a) shows.

Thompson sampling takes a Bayesian route. It keeps a posterior distribution over each arm's expected reward, draws one sample from each posterior in every round, and chooses the arm with the highest sample. For click or completion rewards, each arm's posterior is a Beta distribution: starting from $\text{Beta}(1,1)$, a success increments the first parameter and a failure the second. Arms with uncertain posteriors produce widely spread samples and are chosen occasionally; arms that are clearly worse are rarely chosen. Thompson sampling is old (Thompson, 1933), but its practical strength was appreciated only much later. Chapelle and Li (2011) found it competitive with or better than other bandit algorithms in simulations and on display advertising and news recommendation data from Yahoo. Russo et al. (2018) give a tutorial treatment, including the problems where it performs poorly, such as problems where the most informative action is one the learner should never expect to be best and problems where rewards change over time. Lattimore and Szepesvári (2020) is the standard reference text for the theory of all three algorithms.

16.8.3 Contextual Bandits

Multi-armed bandits learn which arm is best on average. A recommender wants the best arm for each user and request, and that is the setting of contextual bandits, where the learner observes a context x_t of user and request features before choosing an arm. Li et al. (2010) introduced LinUCB for news article recommendation on the Yahoo front page. LinUCB fits, for each arm, a ridge regression of reward on context and chooses the arm with the highest score

$$\hat{\theta}_a^\top x_t + \alpha \sqrt{x_t^\top A_a^{-1} x_t}$$

where $\hat{\theta}_a$ is arm a's estimated coefficient vector, A_a is the regularized matrix of contexts seen for that arm, so that A_a^{-1} measures how uncertain the coefficients are in each direction, and α controls how much exploration the bonus buys. The bonus is large when the current context resembles few contexts previously seen for that arm. Linear Thompson sampling replaces the bonus with a sample from a Gaussian posterior over the coefficients.

Deep rankers raise a practical question, since a linear model on raw features is far weaker than the models of Chapters 6 and 7. The neural-linear approach keeps the deep network for representation and runs a linear bandit on the network's last hidden layer, which gives a usable uncertainty estimate at the cost of one matrix per arm or one shared matrix for the whole model. Su et al. (2024) describe a neural-linear bandit used to explore fresh and tail content in a large-scale recommender. Figure 16.8 compares the policies of this section in a simulation.

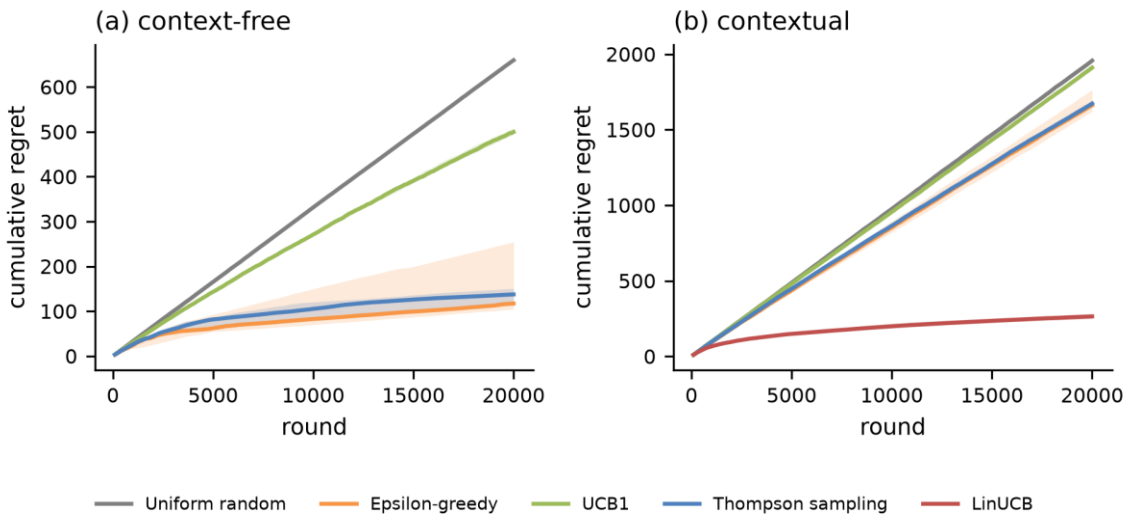


Figure 16.8: Regret of bandit policies, simulated. (a) Cumulative regret over 20,000 rounds on a ten-arm problem with Bernoulli rewards, for uniform random choice, epsilon-greedy, UCB1, and Thompson sampling. Lines show the median across 20 runs, and bands show the interquartile range. With reward rates below 10 percent, UCB1's conservative bonus keeps it exploring, and its regret grows almost linearly. Epsilon-greedy, which explores a fixed 10 percent of the time, ends with the lowest median regret but the widest spread across runs, while Thompson sampling's regret is more consistent and flattens as it learns. (b) Cumulative regret on a contextual problem where the best arm depends on user features, measured against an oracle that knows the best arm for each context. The context-free policies keep accumulating regret at a steady rate, while LinUCB's regret flattens.

16.8.4 Where Bandits Fit in a Recommender

Bandits rarely replace retrieval and ranking; they act at narrow decision points, choosing among a few options the pipeline has already produced. Four uses recur.

The first is presentation choice. Netflix used contextual bandits to choose which artwork to show for each title for each member, an application Chapter 1 described (Chandrashekar et al., 2017). The second is choosing shelves, carousels, or explanations. Spotify described a bandit approach that jointly chose recommendations and their explanations for its home page (McInerney et al., 2018), and Deezer framed carousel personalization as a contextual bandit, compared policies in an offline simulation environment it released for research use, and then tested a subset of Thompson-sampling policies in an online A/B test on its app, which the authors report confirmed the main offline findings (Bendada et al., 2020).

The third use delivers Section 3.4.2's promise for cold-start items: exploration slots, positions reserved for fresh or tail content chosen by an exploration policy. Su et al. (2024) argued that the value of such exploration should be measured by growth of the discoverable corpus, the set of items that users can find through recommendations, and used the user-corpus co-diverted design of Section 16.5.2 to measure it.

The fourth use is the meta-policy that Chapter 1 mentioned. Arms can be whole algorithms, such as a collaborative filter, a popularity baseline, and a sequence model, or whole value-function configurations from Figure 8.2. A bandit allocates traffic among them per context and shifts toward the configuration that earns the most reward, which makes sense when the arms are few, the reward arrives quickly, and the configurations are expected to differ by context or drift over time. For advertising, the same logic applies to new campaigns with no click history, whose predicted click-through rates from Chapter 9 are least reliable until the campaigns have been shown enough to gather evidence.

16.8.5 Bandits Versus A/B Tests

A bandit maximizes cumulative reward while it learns, and an A/B test produces a valid effect estimate for a launch decision. Because a bandit sends more traffic to arms that did well early, a naive comparison of arm means from bandit data is biased: arms with unlucky early results receive fewer later trials and keep their unluckily low means. Bandit logs need the propensity-based methods of Section 16.9.

Bandits suit decisions that repeat per context, with quick feedback and several arms, such as artwork, carousel order, or exploration candidates. A/B tests suit one-time launch decisions judged on long-horizon metrics such as retention. The common combination uses both: the bandit system as a whole becomes the treatment, the static policy it replaces becomes the control, and an A/B test measures whether introducing the bandit improves the metrics that matter. Netflix's artwork work evaluated its bandit policies first by offline replay and then in an A/B test (Chandrashekar et al., 2017).

16.8.6 Engineering Exploration

Microsoft's Decision Service framed contextual bandits as a loop of four steps: explore, which serves a randomized choice; log, which records what happened; learn, which updates a policy from the logs; and deploy, which serves the updated policy (Agarwal et al., 2016). Its authors argued that much of the difficulty lies in plumbing, such as joining each reward to the decision that earned it, and built the service to handle that plumbing once for many applications.

The most important requirement is logging the propensity, the probability with which the policy chose the action it took, at the moment of decision, alongside the context, the action, and later the reward. Without propensities, the logs cannot support the off-policy evaluation of Section 16.9, and the propensities cannot be reliably reconstructed later, because the policy that produced them has since changed.

Exploration has a cost in reward, and Section 15.2.2 suggested treating that cost as deliberate spending from the same error budget that covers releases and experiments. Eligibility and business rules from Section 8.5 still apply to exploratory choices, so an exploration slot must still skip blocked tracks and policy violations. Rewards are often delayed, as Section 16.6.2 discussed, so a bandit that learns from immediate clicks may optimize a proxy at the expense of the goal, and preferences drift, so stationary bandits need discounting or periodic resets. Chapters 2 and 3 cited a small share of traffic as a typical exploration budget. The right budget is a measurement: the reward cost of exploration on a given surface, weighed against the variance that the off-policy estimates of Section 16.9 need.

Companion code. `bandit_ope_simulation.py --mode mab` produces Figure 16.8(a), pitting uniform random choice, epsilon-greedy, UCB1, and Thompson sampling against a simulated catalogue with Bernoulli rewards. `--mode contextual` produces panel (b), adding simulated user features and comparing LinUCB with the context-free policies. Running both, as `--mode mab contextual`, reproduces the full figure. Both modes measure regret only and write no logs. The off-policy evaluation modes in Section 16.9 generate their own logs from an epsilon-greedy logger that records the propensity of every decision.

16.9 Off-Policy Evaluation: Bridging Offline and Online

Logs from a bandit or an exploration slot that record randomized decisions with their propensities can answer counterfactual questions about policies that were never deployed.

16.9.1 The Counterfactual Question

In this section a policy π is any rule that chooses an action given a context: a ranker that chooses which item to place first, a bandit that chooses artwork, or a value-function configuration that chooses a slate. The logging policy π_0 produced the logs, and the target policy π_e is the candidate under evaluation. Off-policy evaluation, often shortened to OPE, estimates how much reward π_e would have earned had it been serving the traffic that π_0 actually served.

Chapter 3 already used propensities. Section 3.3.4 weighted training examples by the inverse probability of their position being examined, following Joachims et al. (2017), and Section 3.3.5 named exploration traffic and inverse propensity scoring as mitigations for exposure bias. Off-policy evaluation applies the same mathematics to the value of a whole decision policy: what would happen if this policy replaced the current one. Gilotte et al. (2018) described the approach at Criteo as offline A/B testing, and Saito and Joachims (2021) survey the methods for recommender settings.

16.9.2 Estimators

Each estimator in this section exists to fix a problem left by the one before it.

Replay is the simplest. Given logs from a uniformly random logging policy, step through the events and keep only those where the target policy would have chosen the same action as the logger; the average reward of the kept events estimates the target policy's value without bias. Li et al. (2011) introduced the method on Yahoo front-page news data and showed that its estimates matched online results. Its problem is waste: with K arms, only about one event in K survives, and the method requires uniform logging.

Inverse propensity scoring, or IPS, uses every event and allows any logging policy whose propensities are known. Each logged reward is weighted by how much more or less likely the target policy is to take the logged action than the logging policy was:

$$\hat{V}_{\text{IPS}}(\pi_e) = \frac{1}{n}\sum_{i=1}^{n} w_i\, r_i, w_i = \frac{\pi_e(a_i \mid x_i)}{\pi_0(a_i \mid x_i)}$$

where x_i, a_i, and r_i are the context, action, and reward of logged event i. Actions that the target policy favours more than the logger did count for more, and actions it would avoid count for less. IPS is unbiased when the propensities are correct and the logger gave every action the target might choose some chance of selection. Bottou et al. (2013) worked through counterfactual reasoning of this kind at scale on the Bing advertising system, using randomization built into ad placement to estimate the effects of changes before running them.

The problem with IPS is variance. When the logger rarely chose an action the target favours, w_i is huge, and a handful of events dominate the estimate. Clipped IPS caps each weight at a maximum value, which reduces variance at the cost of a downward bias for non-negative rewards: capped weights undercount the actions the logger rarely took, and the bias grows as the cap tightens. Self-normalized IPS, or SNIPS, divides by the sum of the weights instead of by n:

$$\hat{V}_{\text{SNIPS}}(\pi_e) = \frac{\sum_i w_i\, r_i}{\sum_i w_i}$$

which keeps the estimate within the range of observed rewards and is more stable than IPS when a few weights are extreme, at the cost of a small bias that shrinks with more data (Swaminathan and Joachims, 2015).

The direct method, or DM, takes a different route entirely. It trains a reward model $\hat{r}(x, a)$ on the logs and then averages the model's predictions for the actions the target policy would choose. It has low variance, since no weights appear, but it inherits every bias of the reward model, and a reward model trained on the logging policy's choices is weakest exactly where the target policy departs from them.

The doubly robust estimator, or DR, combines the two. It starts from the direct method's prediction and adds an importance-weighted correction for the model's errors on the logged actions:

$$\hat{V}_{\mathrm{DR}}(\pi_e) = \frac{1}{n}\sum_{i=1}^{n}\left[\sum_{a}\pi_e\,(a \mid x_i)\hat{r}(x_i, a) + w_i(r_i - \hat{r}(x_i, a_i))\right]$$

If the reward model is accurate, the correction terms are small and variance stays low. If the propensities are accurate, the correction removes the model's bias. The estimator is consistent when either is correct, which is the sense in which it is doubly robust (Dudík et al., 2011). Table 16.3 summarizes the trade-offs.

Table 16.3: Off-policy estimators compared.

Estimator	Idea in one line	Requires	Bias	Variance	Typical failure	Source
Replay	Average the reward over events where the target's choice matches the logged action	Uniformly random logging	Unbiased under uniform logging	High, since most events are discarded	Unusable with non-uniform logs	Li et al. (2011)
IPS	Reweight each logged reward by the ratio of target to logging propensity	Logged propensities; support	Unbiased with correct propensities	High when the policies diverge	A few huge weights dominate the estimate	Bottou et al. (2013)
Clipped IPS	IPS with each weight capped	As IPS, plus a chosen cap	Biased downward; undervalues actions the logger rarely took	Lower than IPS	The choice of cap drives the result	As IPS
SNIPS	Normalize by the sum of the weights instead of by the sample size	As IPS	Small, shrinking with more data	Lower than IPS	Still fails without support	Swaminathan and Joachims

Estimator	Idea in one line	Requires	Bias	Variance	Typical failure	Source
						(2015)
Direct method	Average a learned reward model's predictions over the target's actions	A reward model	Biased if the model is misspecified	Low	Confidently wrong where the model has no data	Dudík et al. (2011)
Doubly robust	Direct method plus a weighted correction of the model's residuals	Propensities and a reward model	Consistent if either part is correct	Moderate	Both parts poor in the same region	Dudík et al. (2011)

16.9.3 Support, Overlap, and Variance

Every estimator except the direct method depends on support, also called overlap: the logging policy must have given positive probability to every action the target policy might take. No estimator can evaluate an action that the logger never took, and the direct method only appears to, by extrapolating its reward model into regions without data. Deterministic logging, where the production ranker always shows its top choice, makes off-policy evaluation of any different ranking impossible. This is why Section 16.8.6 insisted on exploration with logged propensities: exploration is what makes the logs evaluable.

Even with full support, variance grows as the target policy diverges from the logging policy, because the weights grow more extreme. A useful diagnostic is the effective sample size

$$n_{\text{eff}} = \frac{(\sum_i w_i)^2}{\sum_i w_i^2}$$

which equals n when all weights are equal and falls toward one when a single event dominates. An off-policy estimate based on a large log but a small effective sample size is not much better than a guess, and reporting n_{eff} next to every estimate makes this visible. Figure 16.9 shows both effects in a simulation. In panel (a) the direct method overestimates the target policy's value, while the same model fitted on uniformly logged data underestimates it, so even the sign of its bias depends on where the logged data fall.

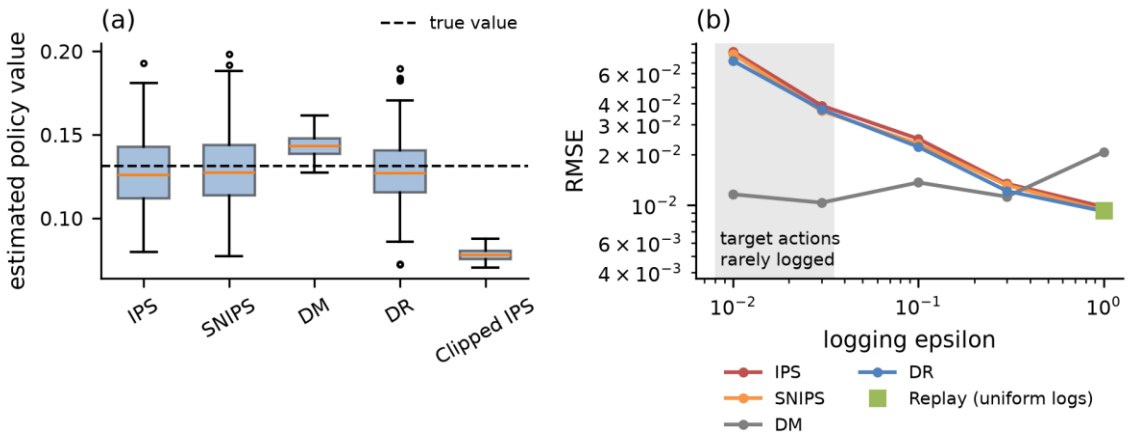


Figure 16.9: Off-policy estimators under bias, variance, and limited support, simulated. (a) Box plots of each estimator's value estimate across 200 simulated logs of 10,000 events, with a dashed line at the target policy's true value, about 0.13: IPS is close to the truth on average but widely spread; clipping each weight at 10 tightens it at the cost of a large downward bias; self-normalization gives a similar spread with little bias; the direct method with a misspecified reward model is tight but off target; and doubly robust removes most of the direct method's bias, at a spread close to that of IPS. (b) Root mean squared error against the logging policy's exploration rate, on logarithmic axes, with replay shown under uniform logging. As exploration falls, the target policy's actions are rarely logged and the error of the weighted estimators grows. The direct method's error comes from its misspecified linear reward model and does not shrink with more exploration; it is largest under uniform logging, where the model is fitted over all contexts, including many in which the target policy would not choose each action. Except under uniform logging, the direct method has the lowest error here, because its bias is small next to the variance of the weighted estimators.

16.9.4 Slates and Rankings

Recommenders choose ordered slates, and the number of possible slates grows combinatorially, so the propensity of any one slate is tiny and whole-slate IPS weights explode. Swaminathan et al. (2017) showed that if a slate's reward is approximately the sum of separate contributions from each item in each position, their pseudoinverse estimator needs far less data than whole-slate IPS, because its weights are built from item-position pairs, which are far more likely to have been logged than any complete slate. The additive assumption fails when items interact, as when a diverse slate is worth more than the sum of its parts, the case Chapter 8's diversity reranking targets. Saito et al. (2021) use a simpler position-based treatment for the Open Bandit Dataset (Section 16.9.5), evaluating each slate position separately.

The same mathematics appears on the learning side. Chen et al. (2019) trained a YouTube recommender with a policy-gradient method on logs from earlier recommenders, using importance weights to correct for the mismatch between logging and learned policies, with a further correction for recommending several items at once. It shares the same dependence on propensities and the same variance problems.

16.9.5 Validating Offline Estimates Against Online Outcomes

Off-policy estimates are only as good as their assumptions, and the assumptions can be checked only against online results. Spotify evaluated a set of playlist recommendation policies both offline, with several off-policy estimators, and online, with A/B tests of the same policies, and examined how well each offline estimator predicted the online results (Gruson et al., 2019). Netflix's artwork work followed

the same two-step pattern, using offline replay to choose among bandit policies before confirming in an A/B test (Chandrashekar et al., 2017).

Public benchmarks support the same caution. Saito et al. (2021) released the Open Bandit Dataset, logs from a fashion e-commerce platform collected under two different logging policies, so that estimators can be evaluated against an on-policy ground truth. Their benchmark experiments found that which estimator performs best depends on the sample size and the setting, so no estimator is a safe default. The dataset and its Open Bandit Pipeline library allow estimators to be tested on real logs.

As with interleaving, off-policy evaluation screens candidates for online traffic, and an online test confirms. Recording how off-policy estimates compare with later A/B results builds the offline-online correlation record that Section 3.4.1 recommended, which shows over time how far the estimates can be trusted.

Companion code. Run `bandit_ope_simulation.py --mode ope` to evaluate a target policy from logs produced by an epsilon-greedy logger, comparing IPS, clipped IPS, SNIPS, the direct method, and doubly robust estimates with the true value from the simulator. `--mode support` lowers the logger's exploration rate step by step, reports the effective sample size and the error of IPS, SNIPS, the direct method, and doubly robust, and adds replay under uniform logging, where it applies. Running both, as `--mode ope support`, reproduces Figure 16.9, on simulated data.

16.10 Choosing a Method and Building an Experimentation Practice

The methods of this chapter differ most in what they randomize and when: nothing at all, randomization from the past reused, randomization within each impression, fixed randomization between units, adaptive randomization, or randomization extended over months. Table 16.4 condenses them into a single comparison, with a final row that marks the boundary with Chapter 15.

16.10.1 Matching the Question to the Method

Table 16.4: Choosing an evaluation method.

Question	Method	What is randomized	Traffic and time	Risk to users	Cannot tell you	Section
Does the candidate rank held-out interactions better?	Offline metrics on logs	Nothing	None	None	Whether the gain survives the change in exposure, or reaches business metrics	Ch 3; 16.2.2
What would a target policy have earned on logged traffic?	Off-policy evaluation	Past exploration by the logging policy	None at evaluation time	None at evaluation time	The value of actions the logger rarely or never took	16.9

Question	Method	What is randomized	Traffic and time	Risk to users	Cannot tell you	Section
Which of two rankers do users prefer?	Interleaving	Team assignment within each impression	Low	Low	The size of the effect, or any whole-page or long-term effect	16.7
Does the change move user and business metrics, and by how much?	User-randomized A/B test	Users or other units, fixed for the test	Moderate to high	Moderate	Effects that leak between arms, or that emerge after the test ends	16.3; 16.4
The same question, when units share a model, inventory, budget, or network	Interference-aware design	Clusters, listings, budgets, time blocks, or corpus partitions	High	Moderate	Leakage beyond the randomized blocks	16.5
Which option is best for each context, learned while serving?	Bandit allocation	Arms, adaptively per decision	Ongoing	Low to moderate	An unbiased effect estimate from a simple comparison of arm means	16.8
Does the effect persist, grow, or reverse?	Long-term holdback	Users kept on the old experience after launch	Small traffic, long duration	Opportunity cost for held-back users	A quick answer	16.6.2
Is this release safe to keep rolling out?	Canary and release gates	A small slice of traffic	Short	Low	Whether the new version is better	Ch 15 (15.9.5)

The rows work best as questions asked in order. Release safety comes first, because a change that cannot safely keep rolling out never reaches a comparison, and Chapter 15 answers that question. Then ask whether two rankers are being compared on one surface, whether the decision repeats per context with quick feedback, whether randomized logs with adequate support exist, whether units share a model, inventory, budget, or network, and whether the outcome arrives slowly. Every A/B test still needs the basics of Sections 16.3 and 16.4. Several answers can only be measured on a given surface: how well interleaving and off-policy estimates agree with later A/B results, how large interference bias is, and how much CUPED helps.

16.10.2 Platform Essentials

Most of these practices are hard to sustain by hand, and teams that experiment at scale build them into a shared platform. Assignment should use salted hashing with layers, so that many experiments run concurrently without colliding (Tang et al., 2010). Analysis should apply the correct variance by default, using the delta method for ratio metrics and CUPED where pre-period data exist, so that no analyst has to remember to. Sample ratio checks and guardrail metrics should run automatically on every experiment and block the readout when they fail. An experiment registry and decision log should record each hypothesis, design, metric, analysis plan, and decision, which turns individual tests into an institutional memory and makes it possible to compute later how often offline gains survived online. Holdback populations need management, with clear ownership and expiry dates (Kohavi et al., 2020). Propensity logging belongs in the serving platform, shared by every team that randomizes, so that every randomized decision is evaluable later (Agarwal et al., 2016). And null results deserve the same reporting as wins, for the reason Section 16.4.6 gave: only a full record shows how many tests were run, and so how many significant results to expect by chance (Tingley et al., 2021b).

16.10.3 Boundaries

Candidates that come from automated research loops, such as the agentic loop of Appendix D that proposes and evaluates candidates offline, face exactly the same evidence ladder as candidates proposed by people, and offline success from an automated loop earns a candidate online traffic and nothing more. Release safety remains Chapter 15's responsibility, so an experiment's verdict about whether a change is better never substitutes for the canary's verdict about whether it is safe.

16.11 Chapter Summary

Launch decisions need causal estimates of what a change will do to users, and randomized experiments supply them by making the treatment independent of everything else about the user. A trustworthy experiment starts with a pre-registered decision rule, a well-chosen randomization unit, layered assignment by salted hashing, counterfactual logging for triggered analysis, and metrics in explicit roles, and A/A tests and sample ratio checks confirm the machinery before anyone reads a result.

Required sample size grows with metric variance and with the inverse square of the effect, so heavy-tailed metrics benefit from capping, ratio metrics need the delta method, and pre-period data reduce variance through CUPED and CUPAC. Repeated looks need group sequential or always-valid methods, and many metrics or variants need explicit corrections.

Recommenders break the no-interference assumption through shared models, inventory, budgets, and social ties. Cluster, two-sided, budget-split, switchback, data-diverted, and user-corpus co-diverted designs each contain one channel at a cost. Novelty and primacy distort short tests, and users keep learning for months, so long-term holdbacks and surrogate indices are needed to measure lasting effects.

Interleaving credits clicks by team within each list, which makes it far more sensitive than an A/B test for choosing a ranker, though it cannot measure effect size or whole-page effects. Bandits trade reward now for information later and fit narrow, repeated decisions. Their logged propensities enable off-policy evaluation of policies never deployed, provided every action a candidate policy would take had some chance of being logged. Off-policy estimates screen candidates, and online tests confirm them.

The companion code, described in its README, runs the A/B simulations on Yambda or synthetic data and the interleaving, bandit, and off-policy simulations on synthetic data, with known true effects throughout.

16.12 The Recommender as a Learning System

Experimentation depends on almost everything that came before in the book. A trustworthy experiment needs clean logs, a clear objective, a pipeline whose stages can be changed independently, and a serving system that records what it did and why. Five ideas run through that dependence and through the book as a whole.

The first idea is that a recommender is a decision system. Models produce predictions, such as probabilities of a click, a completion, or a purchase, but users never see predictions. They see decisions: which items appear, in which order, next to which explanations, under which business rules. The step from prediction to decision passes through a value function that weighs competing objectives, constraints that encode policy, and rules that encode commitments to partners and users (Chapter 8). Every one of those choices is a statement about what the system is for. An objective that is chosen deliberately, examined, and tested keeps a system serving its stated goals; an objective inherited from whichever loss function was convenient lets it drift toward whatever is easiest to measure.

The second idea is that the data is produced by the system itself. A recommender learns from interactions with the items it chose to show, in the positions it chose to show them, so every training example carries the imprint of an earlier model's decisions. Position bias, exposure bias, popularity bias, and feedback loops are all consequences of that circularity (Chapter 3). Randomized experiments and deliberate exploration are the only evidence a recommender collects that its own past decisions did not shape, which is why logging propensities and preserving randomized traffic are investments in the system's ability to learn at all.

The third idea is that evidence comes in grades. An offline metric, an off-policy estimate, an interleaving preference, an A/B test result, and a long-term holdback each answer a different question with different costs and different failure modes. Offline metrics are cheap and plentiful but blind to the effect of changing what users see; holdbacks are slow and costly but capture what users learn over months. The discipline lies in matching each question to the least expensive evidence that can answer it reliably, and in recording, over time, how well the cheaper grades predicted the more expensive ones.

The fourth idea is that systems constraints are modelling constraints. A model that cannot score candidates within the latency budget, that needs features which cannot be computed fresh at request time, or that behaves differently in serving than in training is not a candidate, however well it scores offline (Chapters 4, 13, and 14). Retrieval exists because ranking every item is too slow; feature stores exist because training and serving must see the same values; canaries exist because a model file can fail in ways no offline test anticipated. Architecture decides which models are possible, so modelling and engineering are one design problem.

The fifth idea is that a recommender is never finished. Users change, catalogues change, and the system's own recommendations change the behaviour it learns from. Monitoring detects when the world has moved, retraining incorporates the new evidence, and experimentation decides which changes deserve to stay (Chapter 15). Together they form a loop by which the system improves, and the quality of a recommender depends as much on the speed and reliability of that loop as on the model at its centre.

Several problems remain open. Measuring long-term value is still difficult: surrogates and holdbacks help, yet the outcomes that matter most, such as whether recommendations enrich users' lives or narrow them, are slow, diffuse, and hard to define. Interference through shared models has been documented, but designs that contain it at an acceptable cost are still being developed. Generative and large-language-model-based recommenders, discussed in Chapter 12, raise the hardest evaluation questions of all. When a system can generate an open-ended space of responses, explanations, and item combinations, the support assumption behind off-policy evaluation is nearly impossible to satisfy, interleaving has no natural way to merge two generated answers, and the gap between offline judgement and online behaviour may be wider than for any earlier generation of models.

Those problems share a common shape, and so does every method in this book. A recommender is a system that acts on incomplete knowledge of its users, observes the consequences of its own actions, and must turn those observations into better actions; its quality is ultimately determined by how honestly it can tell what its decisions actually caused.

References

Agarwal, A., Bird, S., Cozowicz, M., Hoang, L., Langford, J., Lee, S., Li, J., Melamed, D., Oshri, G., Ribas, O., Sen, S., and Slivkins, A. (2016). "Making Contextual Decisions with Low Technical Debt." arXiv preprint arXiv:1606.03966. https://arxiv.org/abs/1606.03966

Athey, S., Chetty, R., Imbens, G. W., and Kang, H. (2026). "The Surrogate Index: Combining Short-Term Proxies to Estimate Long-Term Treatment Effects More Rapidly and Precisely." Review of Economic Studies 93(4). Oxford University Press. https://doi.org/10.1093/restud/rdaf087

Auer, P., Cesa-Bianchi, N., and Fischer, P. (2002). "Finite-time Analysis of the Multiarmed Bandit Problem." Machine Learning 47(2-3). Springer. https://doi.org/10.1023/A:1013689704352

Bendada, W., Salha, G., and Bontempelli, T. (2020). "Carousel Personalization in Music Streaming Apps with Contextual Bandits." Proceedings of the 14th ACM Conference on Recommender Systems. ACM. https://doi.org/10.1145/3383313.3412217

Bernardi, L., Mavridis, T., and Estevez, P. (2019). "150 Successful Machine Learning Models: 6 Lessons Learned at Booking.com." Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining. ACM. https://doi.org/10.1145/3292500.3330744

Bibaut, A., Chou, W., Ejdemyr, S., and Kallus, N. (2024). "Improve Your Next Experiment by Learning Better Proxy Metrics From Past Experiments." Netflix Technology Blog. https://netflixtechblog.com/improve-your-next-experiment-by-learning-better-proxy-metrics-from-past-experiments-64c786c2a3ac

Bojinov, I., Simchi-Levi, D., and Zhao, J. (2023). "Design and Analysis of Switchback Experiments." Management Science 69(7). INFORMS. https://doi.org/10.1287/mnsc.2022.4583

Bottou, L., Peters, J., Quiñonero-Candela, J., Charles, D. X., Chickering, D. M., Portugaly, E., Ray, D., Simard, P., and Snelson, E. (2013). "Counterfactual Reasoning and Learning Systems: The Example of Computational Advertising." Journal of Machine Learning Research 14. JMLR. https://jmlr.org/papers/v14/bottou13a.html

Brennan, J., Cong, Y., Yu, Y., Lin, L., Peng, Y., Meng, C., Han, N., Pouget-Abadie, J., and Holtz, D. (2025). "Reducing Symbiosis Bias through Better A/B Tests of Recommendation Algorithms." Proceedings of the ACM Web Conference 2025. ACM. https://doi.org/10.1145/3696410.3714738

Chandrashekar, A., Amat, F., Basilico, J., and Jebara, T. (2017). "Artwork Personalization at Netflix." Netflix Technology Blog. https://netflixtechblog.com/artwork-personalization-at-netflix-c589f074ad76

Chapelle, O., Joachims, T., Radlinski, F., and Yue, Y. (2012). "Large-Scale Validation and Analysis of Interleaved Search Evaluation." ACM Transactions on Information Systems 30(1). ACM. https://doi.org/10.1145/2094072.2094078

Chapelle, O., and Li, L. (2011). "An Empirical Evaluation of Thompson Sampling." Advances in Neural Information Processing Systems 24. Curran Associates. https://proceedings.neurips.cc/paper_files/paper/2011/hash/e53a0a2978c28872a4505bdb51db06dc-Abstract.html

Chen, M., Beutel, A., Covington, P., Jain, S., Belletti, F., and Chi, E. H. (2019). "Top-K Off-Policy Correction for a REINFORCE Recommender System." Proceedings of the Twelfth ACM International Conference on Web Search and Data Mining. ACM. https://doi.org/10.1145/3289600.3290999

Deng, A., Knoblich, U., and Lu, J. (2018). "Applying the Delta Method in Metric Analytics: A Practical Guide with Novel Ideas." Proceedings of the 24th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining. ACM. https://doi.org/10.1145/3219819.3219919

Deng, A., and Shi, X. (2016). "Data-Driven Metric Development for Online Controlled Experiments: Seven Lessons Learned." Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining. ACM. https://doi.org/10.1145/2939672.2939700

Deng, A., Xu, Y., Kohavi, R., and Walker, T. (2013). "Improving the Sensitivity of Online Controlled Experiments by Utilizing Pre-Experiment Data." Proceedings of the Sixth ACM International Conference on Web Search and Data Mining. ACM. https://doi.org/10.1145/2433396.2433413

Dmitriev, P., Gupta, S., Kim, D. W., and Vaz, G. (2017). "A Dirty Dozen: Twelve Common Metric Interpretation Pitfalls in Online Controlled Experiments." Proceedings of the 23rd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining. ACM. https://doi.org/10.1145/3097983.3098024

Dudík, M., Langford, J., and Li, L. (2011). "Doubly Robust Policy Evaluation and Learning." Proceedings of the 28th International Conference on Machine Learning. Omnipress. https://arxiv.org/abs/1103.4601

Fabijan, A., Gupchup, J., Gupta, S., Omhover, J., Qin, W., Vermeer, L., and Dmitriev, P. (2019). "Diagnosing Sample Ratio Mismatch in Online Controlled Experiments: A Taxonomy and Rules of Thumb for Practitioners." Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining. ACM. https://doi.org/10.1145/3292500.3330722

Gilotte, A., Calauzènes, C., Nedelec, T., Abraham, A., and Dollé, S. (2018). "Offline A/B Testing for Recommender Systems." Proceedings of the Eleventh ACM International Conference on Web Search and Data Mining. ACM. https://doi.org/10.1145/3159652.3159687

Gruson, A., Chandar, P., Charbuillet, C., McInerney, J., Hansen, S., Tardieu, D., and Carterette, B. (2019). "Offline Evaluation to Make Decisions About Playlist Recommendation Algorithms." Proceedings of the Twelfth ACM International Conference on Web Search and Data Mining. ACM. https://doi.org/10.1145/3289600.3291027

Hohnhold, H., O'Brien, D., and Tang, D. (2015). "Focusing on the Long-term: It's Good for Users and Business." Proceedings of the 21st ACM SIGKDD International Conference on Knowledge Discovery and Data Mining. ACM. https://doi.org/10.1145/2783258.2788583

Jackson, S. (2018). "How Booking.com Increases the Power of Online Experiments with CUPED." Booking.com Data Science. https://booking.ai/how-booking-com-increases-the-power-of-online-experiments-with-cuped-995d186fff1d

Jiang, R., Chiappa, S., Lattimore, T., György, A., and Kohli, P. (2019). "Degenerate Feedback Loops in Recommender Systems." Proceedings of the 2019 AAAI/ACM Conference on AI, Ethics, and Society. ACM. https://doi.org/10.1145/3306618.3314288

Joachims, T., Swaminathan, A., and Schnabel, T. (2017). "Unbiased Learning-to-Rank with Biased Feedback." Proceedings of the Tenth ACM International Conference on Web Search and Data Mining. ACM. https://arxiv.org/abs/1608.04468

Johari, R., Koomen, P., Pekelis, L., and Walsh, D. (2017). "Peeking at A/B Tests: Why It Matters, and What to Do About It." Proceedings of the 23rd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining. ACM. https://doi.org/10.1145/3097983.3097992

Johari, R., Li, H., Liskovich, I., and Weintraub, G. Y. (2022). "Experimental Design in Two-Sided Platforms: An Analysis of Bias." Management Science 68(10). INFORMS. https://doi.org/10.1287/mnsc.2021.4247

Kohavi, R., Tang, D., and Xu, Y. (2020). Trustworthy Online Controlled Experiments: A Practical Guide to A/B Testing. Cambridge University Press. https://doi.org/10.1017/9781108653985

Lattimore, T., and Szepesvári, C. (2020). Bandit Algorithms. Cambridge University Press. https://doi.org/10.1017/9781108571401

Li, J. (2020). "Improving Experimental Power through Control Using Predictions as Covariate (CUPAC)." DoorDash Engineering Blog. https://careersatdoordash.com/blog/improving-experimental-power-through-control-using-predictions-as-covariate-cupac/

Li, L., Chu, W., Langford, J., and Schapire, R. E. (2010). "A Contextual-Bandit Approach to Personalized News Article Recommendation." Proceedings of the 19th International Conference on World Wide Web. ACM. https://doi.org/10.1145/1772690.1772758

Li, L., Chu, W., Langford, J., and Wang, X. (2011). "Unbiased Offline Evaluation of Contextual-bandit-based News Article Recommendation Algorithms." Proceedings of the Fourth ACM International Conference on Web Search and Data Mining. ACM. https://doi.org/10.1145/1935826.1935878

Liu, M., Mao, J., and Kang, K. (2021). "Trustworthy and Powerful Online Marketplace Experimentation with Budget-split Design." Proceedings of the 27th ACM SIGKDD Conference on Knowledge Discovery and Data Mining. ACM. https://doi.org/10.1145/3447548.3467193

McInerney, J., Lacker, B., Hansen, S., Higley, K., Bouchard, H., Gruson, A., and Mehrotra, R. (2018). "Explore, Exploit, and Explain: Personalizing Explainable Recommendations with Bandits." Proceedings of the 12th ACM Conference on Recommender Systems. ACM. https://doi.org/10.1145/3240323.3240354

Parks, J., Aurisset, J., and Ramm, M. (2017). "Innovating Faster on Personalization Algorithms at Netflix Using Interleaving." Netflix Technology Blog. https://netflixtechblog.com/interleaving-in-online-experiments-at-netflix-a04ee392ec55

Radlinski, F., Kurup, M., and Joachims, T. (2008). "How Does Clickthrough Data Reflect Retrieval Quality?" Proceedings of the 17th ACM Conference on Information and Knowledge Management. ACM. https://doi.org/10.1145/1458082.1458092

Russo, D. J., Van Roy, B., Kazerouni, A., Osband, I., and Wen, Z. (2018). "A Tutorial on Thompson Sampling." Foundations and Trends in Machine Learning 11(1). Now Publishers. https://doi.org/10.1561/2200000070

Sadeghi, S., Gupta, S., Gramatovici, S., Lu, J., Ai, H., and Zhang, R. (2022). "Novelty and Primacy: A Long-Term Estimator for Online Experiments." Technometrics 64(4). Taylor and Francis. https://doi.org/10.1080/00401706.2022.2124309

Saint-Jacques, G., Varshney, M., Simpson, J., and Xu, Y. (2019). "Using Ego-Clusters to Measure Network Effects at LinkedIn." arXiv preprint arXiv:1903.08755. https://arxiv.org/abs/1903.08755

Saito, Y., Aihara, S., Matsutani, M., and Narita, Y. (2021). "Open Bandit Dataset and Pipeline: Towards Realistic and Reproducible Off-Policy Evaluation." Proceedings of the Neural Information Processing Systems Track on Datasets and Benchmarks. https://arxiv.org/abs/2008.07146

Saito, Y., and Joachims, T. (2021). "Counterfactual Learning and Evaluation for Recommender Systems: Foundations, Implementations, and Recent Advances." Proceedings of the 15th ACM Conference on Recommender Systems. ACM. https://doi.org/10.1145/3460231.3473320

Schultzberg, M., and Ankargren, S. (2023). "Choosing a Sequential Testing Framework: Comparisons and Discussions." Spotify Engineering. https://engineering.atspotify.com/2023/03/choosing-sequential-testing-framework-comparisons-and-discussions

Si, N. (2023). "Tackling Interference Induced by Data Training Loops in A/B Tests: A Weighted Training Approach." arXiv preprint arXiv:2310.17496. https://arxiv.org/abs/2310.17496

Su, Y., Wang, X., Le, E. Y., Liu, L., Li, Y., Lu, H., Lipshitz, B., Badam, S., Heldt, L., Bi, S., Chi, E. H., Goodrow, C., Wu, S.-L., Baugher, L., and Chen, M. (2024). "Long-Term Value of Exploration: Measurements, Findings and Algorithms." Proceedings of the 17th ACM International Conference on Web Search and Data Mining. ACM. https://doi.org/10.1145/3616855.3635833

Swaminathan, A., and Joachims, T. (2015). "The Self-Normalized Estimator for Counterfactual Learning." Advances in Neural Information Processing Systems 28. Curran Associates. https://proceedings.neurips.cc/paper_files/paper/2015/hash/39027dfad5138c9ca0c474d71db915c3-Abstract.html

Swaminathan, A., Krishnamurthy, A., Agarwal, A., Dudík, M., Langford, J., Jose, D., and Zitouni, I. (2017). "Off-policy Evaluation for Slate Recommendation." Advances in Neural Information Processing Systems 30. Curran Associates. https://arxiv.org/abs/1605.04812

Tang, D., Agarwal, A., O'Brien, D., and Meyer, M. (2010). "Overlapping Experiment Infrastructure: More, Better, Faster Experimentation." Proceedings of the 16th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining. ACM. https://doi.org/10.1145/1835804.1835810

Thompson, W. R. (1933). "On the Likelihood that One Unknown Probability Exceeds Another in View of the Evidence of Two Samples." Biometrika 25(3-4). Oxford University Press. https://doi.org/10.1093/biomet/25.3-4.285

Tingley, M., Zheng, W., Ejdemyr, S., Lane, S., and McFarland, C. (2021a). "Interpreting A/B Test Results: False Negatives and Power." Netflix Technology Blog. https://netflixtechblog.com/interpreting-a-b-test-results-false-negatives-and-power-6943995cf3a8

Tingley, M., Zheng, W., Ejdemyr, S., Lane, S., Lindon, M., and McFarland, C. (2021b). "Building Confidence in a Decision." Netflix Technology Blog. https://netflixtechblog.com/building-confidence-in-a-decision-8705834e6fd8

Ugander, J., Karrer, B., Backstrom, L., and Kleinberg, J. (2013). "Graph Cluster Randomization: Network Exposure to Multiple Universes." Proceedings of the 19th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining. ACM. https://doi.org/10.1145/2487575.2487695

Xie, H., and Aurisset, J. (2016). "Improving the Sensitivity of Online Controlled Experiments: Case Studies at Netflix." Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining. ACM. https://doi.org/10.1145/2939672.2939733

Zhang, Q., Du, M., Andersen, R., and He, L. (2022). "Beyond A/B Test: Speeding up Airbnb Search Ranking Experimentation through Interleaving." The Airbnb Tech Blog. https://medium.com/airbnb-engineering/beyond-a-b-test-speeding-up-airbnb-search-ranking-experimentation-through-interleaving-7087afa09c8e

Zhao, Z., and Feng, E. (2017). "Building an Intelligent Experimentation Platform with Uber Engineering." Uber Engineering Blog. https://www.uber.com/us/en/blog/experimentation-platform/